

Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN

# Análise, deseño e implementación dunha gramola virtual.

**Estudante:** Carlos Mañoso Sierra
**Dirección:** Diego Andrade Canosa

A Coruña, November de 2021.

*Para todas as persoas que desexan escoitar a súa canción favorita.*

**Agradecementos**

A todos eses que desexaban ver este proxecto rematado, especialmente a Cris e a Sergio.

**Resumo**

En calquera situación na que se escoite música, sempre hai alguén que prefire un estilo, unha canción, ou unha lista de reprodución ao completo. Para resolver este problema na década dos 60 comezaron a introducirse as gramolas, deste xeito arranxabase o problema, pero eran aparellos grandes e nos que tan só se podían escoitar os éxitos do momento e nos lugares nos que existían estes reprodutores. Hoxe en día a maneira de escoitar música sufriu un cambio grande, a maioría reprodúcese a través de Internet, dende plataformas coma pode ser Spotify. Isto aumentou a variedade musical e simplificou o poder escoitar música en calquera lugar, é estraño unha festa con amigos sen música, polo tanto son frecuentes as persoas que piden escoitar a súa canción favorita.
O obxectivo deste proxecto é resolver este problema recorrendo a mesma idea que antano, actualizado a forma de escoitar música hoxe en día. Crearase unha aplicación web cun backend en Django Rest Framework capaz de manexar pola súa conta as cancións na cola de reprodución e un frontend en Ionic con Angular para a iteración cos usuarios dende a web.
Obtivemos unha aplicación que xestiona unha lista de reprodución compartida, capaz de reproducila en calquera aparello que incorpore Spotify coma a súa plataforma de música. Nesta, dunha maneira moi sinxela os usuarios poderán crear unha sesión para compartir, conectala co seu Spotify e deixar que o resto dos usuarios engadan as cancións que desexen e voten polas que mais lles gustan.

**Abstract**

In any situation where music is listened to, there is always someone who prefers a style, a song or a compleat playlist. To solve this kind of problem in the 60s the jukeboxes appeared, this way the problem was solved, but the machines were big, you could listen just the hits of the moment and where the device were. Nowadays how we listen to music changed a lot, usually the music is played throght the Internet, from platforms like Spotify. This increased the music variety and made easier to listen music anywhere, it's weird a party with friends without music, so is usual the people that asks to listen their favourite song.
The objetive of the project it's to solve this problem the same way yesteryear, updated to the way we listen music nowadays. We will build a web application with a Django Rest Framework backend able to manage the songs at the queue and a frontend in Ionic with Angular, so the users can interact with the web.
We got an app that manages a shared playlist, able of play it at any device with Spotify as

music platform. In a very simple way the users will be able to create a session to share, connect it with their Spotify and let the other users add songs or vote for their favourite ones.

**Palabras chave:**

- Gramola
- Música
- Bar
- Cancións
- Spotify
- Web app
- Ionic
- Django

**Keywords:**

- Jukebox
- Music
- Bar
- Songs
- Spotify
- Web app
- Ionic
- Django

# Índice Xeral


**1 Introdución** **1**

1.1 Obxectivos . . . 2

1.2 Estrutura da memoria . . . 2

**2 Análise** **5**

2.1 Viabilidade económica . . . 5

2.2 Viabilidade técnica . . . 6

2.3 Viabilidade do mercado . . . 6

2.4 Análise . . . 7

2.5 Actores . . . 9

2.6 Requisitos da aplicación . . . 11

2.6.1 Requisitos funcionais . . . 11

2.6.2 Requisitos non funcionais . . . 11

**3 Tecnoloxías** **13**

3.1 Ferramentas de desenvolvemento . . . 13

3.2 Frameworks e librarías . . . 15

3.3 Linguaxes . . . 15

3.4 Sistema de xestión de base de datos . . . 17

3.5 APIs . . . 17

3.6 Estado da arte . . . 18

**4 Planificación** **25**

4.1 Metodoloxía . . . 25

4.2 Scrum . . . 25

4.2.1 Artefactos Scrum . . . 26

4.2.2 Protocolos ou eventos de scrum . . . 27

4.2.3 Roles . . . 28

4.2.4 Scrum no proxecto . . . 29
4.3 Planificación inicial . . . 29
4.4 Adaptación da planificación a Scrum . . . 30
4.4.1 Primeiro sprint . . . 30
4.4.2 Segundo sprint . . . 30
4.4.3 Terceiro sprint . . . 33
4.4.4 Cuarto sprint . . . 33
4.4.5 Quinto sprint . . . 33
4.5 Custos do proxecto . . . 34

**5 Deseño** **37**
5.1 Lóxica de funcionamento . . . 37
5.2 Arquitectura . . . 38
5.2.1 MVC: Modelo, vista, controlador . . . 38
5.2.2 MTV: Modelo, plantilla, vista . . . 40
5.2.3 Esquema xeral . . . 41
5.3 Frontend . . . 41
5.3.1 Arquitectura do frontend . . . 42
5.3.2 Vista . . . 43
5.3.3 Controller . . . 44
5.4 Backend . . . 45
5.4.1 Arquitectura do backend . . . 45
5.4.2 Modelo . . . 45
5.4.3 Vista . . . 46

**6 Implementación** **49**
6.1 Estrutura do proxecto . . . 49
6.2 Percorrido das peticións . . . 52
6.3 Seguridade . . . 53
6.4 Desenvolvemento . . . 56
6.4.1 Frontend . . . 56
6.4.2 Backend . . . 65
6.5 Backend . . . 70
6.5.1 Modelo . . . 70
6.5.2 Plantilla . . . 70
6.5.3 Vista . . . 70
6.6 Instruccións de execución . . . 76

**7 Conclusións e traballo futuro** **77**
7.1 Conclusións . . . . . 77
7.2 Traballo futuro . . . . . 77

**Relación de Acrónimos** **81**

**Glosario** **83**

**Bibliografía** **85**

# Índice de Figuras


2.1 Diagrama de actores . . . . . 10

3.1 TouchTunes . . . . . 19
3.2 Ami Entertainment . . . . . 19
3.3 Jukestar App . . . . . 20
3.4 Outloud App . . . . . 22
3.5 Captura dunha sala en Jukebox Today . . . . . 22
3.6 Raveberry Server . . . . . 23

4.1 Ciclo de sprint en Scrum . . . . . 26
4.2 Roles en Scrum . . . . . 28
4.3 Mockups do frontend . . . . . 31

5.1 Model, controller, view. . . . . 39
5.2 Model, template, view. . . . . 40
5.3 Estrutura da base de datos . . . . . 46

6.1 Estrutura do proxecto . . . . . 50
6.2 Estrutura do backend . . . . . 51
6.3 Estrutura do frontend . . . . . 52
6.4 Percorrido dunha petición. . . . . 54
6.5 Vista en diferentes dispositivos . . . . . 57

# Índice de Táboas

2.1 Custos dos plans de Spotify . . . . . 6

4.1 Custos do proxecto . . . . . 34
4.2 Tempo empregado no proxecto . . . . . 35

# Capítulo 1
# Introdución

Neste primeiro capítulo da memoria, se explica á motivación, se expoñen os obxectivos e se define á estrutura.

Durante a décadas dos 60 e 70 a industria musical sufriu unha revolución, nunha gran parte dos lugares de restauración comezaron aparecer aparellos que permitían elixir as seguintes cancións no local, as gramolas. Coa dixitalización da música, as gramolas foron desaparecendo e en poucos casos adaptándose os novos formatos e dixitalizandose. Nos nosos días, coas múltiples plataformas que existen e a música na nube é moi complicado atopar un bar no que se permita elixir a seguinte canción, coma antes. Do mesmo xeito en calquera lugar onde soe música sempre hai alguén que quere engadir a súa canción a cola de reprodución, este pode ser un obxectivo sinxelo ou mais complicado dependendo de moitos factores. Con este proxecto tratamos de resolver estes problemas e dun xeito doado compartir os diferentes gustos musicais.

O obxectivo deste proxecto é crear unha aplicación web na que os usuarios poidan engadir as cancións que desexen a unha lista de reprodución compartida e votar polas súas favoritas. Deste xeito as cancións reprodúcense segundo a orde de chegada e segundo o seu número de votos. Polo tanto a idea é crear unha gramola virtual na que os usuarios non teñen que recorrer a ningún aparello, con calquera dispositivo con acceso a Interet deberían de ser capaces de conectarse a gramola e engadir cancións. Para realizar a reprodución de música usase o cliente de Spotify que se controla mediante a súa propia API deste xeito se xestiona a música que vai soar, polo tanto poderá ser reproducida en calquera dispositivo que teña un cliente desta plataforma. Evítase interromper o uso normal do Spotify, é dicir, facendo que o usuario poida usar o seu cliente de Spotify coma sempre e facendo que a aplicación engada as cancións a cola de reprodución tan so nos momentos necesarios. Co propósito de facilitar o seu uso e desta maneira incrementar o número de usuarios creouse a plataforma do xeito mais intuitivo posible para estes.

Na parte técnica o obxectivo é crear una Progressive web application (PWA), é dicir unha

aplicación creada coas tecnoloxías web, que funciona en calqueira tipo de navegador compatible. A aplicación constará dun backend en Django Rest Framework, que será o encargado de xestionar as solicitudes a API de Spotify, e manexar as propias peticións do noso frontend en Ionic que utiliza Angular coma framework, en canto a base de datos utilizaremos SQLite para o almaceamento dos usuarios e as listas de reprodución.

## 1.1 Obxectivos

O proxecto pretende crear unha plataforma de creación de listas de reprodución de música de forma colaborativa dunha forma similar a unha gramola e a reprodución destas mediante un cliente de Spotify. Os obxetivos funcionais son os seguintes:

- Xestión de sesións nas que os administradores poderán conectar a súa conta de Spotify para a reprodución de música.
- Creación de novos usuarios dentro de cada unha das sesións, contando estas cun usuario administrador e os demáis usuarios rexistrados.
- Posibilidade de facer unha búsqueda de cancións para engadir a cola de reprodución.
- Opción de votación para as cancións na cola de reprodución, isto permitirá cambiar a orde na que as cancións son reproducidas.

Do mesmo xeito expoñemos os obxetivos non funcionais:

- As comunicacións tanto coa API propia coma coa de Spotify deberán de ser seguras.
- A usabilidade do sistema ten que ser sinxela, tanto o rexistro coma a búsqueda de música ten que ser clara.
- Funcionamento sen manter a aplicación aberta por parte do administrador.
- A versión web deberá ser adaptable, tanto a pantallas grandes coma a móbiles.

## 1.2 Estrutura da memoria

Este documento estruturase en 7 capítulos.

- Neste primeiro capítulo describese a motivación, e os obxectivos do proxecto.
- No capítulo segundo, contén o análise de viabilidade do proxecto.
- No capítulo terceiro explícanse as tecnoloxías utilizadas durante o proceso de desenvolvemento.

- No capítulo 4.
- No capítulo 5.
- No capítulo 6.
- No capítulo 7.

Capítulo 2

# Análise

Neste segundo capítulo se fará unha análise da viabilidade do proxecto, deste xeito poderemos determinar se é posible que chegue a finalizarse con éxito.

Para determinar se o proxecto poderá acadar o seu obxectivo existen varios aspectos a analizar, o conxunto destes é a viabilidade do proxecto. En xeral para os proxectos software as áreas nas que recompilar información para determinar a viabilidade son: técnica, económica e de mercado, deste xeito pódese saber de maneira anticipada cal vai ser o custo aproximado do proxecto, o tempo necesario para rematalo e cal é o impacto no mercado esperado, a continuación descríbense estes diferentes dominios.

## 2.1 Viabilidade económica

No análise económico téñense en conta tres custos principais, os custos en recursos humanos, en software e en hardware.

Para os **recursos humanos**, téñense en conta a cantidade de persoas necesarias e o custo por hora de cada unha destas. Neste caso é unha soa persoa a encargada de facer o deseño, implementación e probas, tampouco é necesario ter en conta o custo por hora polo tanto non existe un custo real monetario en recursos humanos para a realización do proxecto.

Os **recursos software** teñen en conta os custos do software utilizado para o desenvolvemento do proxecto. Neste proxecto utilizase software libre e gratuíto na maior medida posible, partindo do sistema operativo ata os frameworks e librarías, pasando polas ferramentas de desenvolvemento coma o editor de texto ou o navegador coma veremos mais adiante 3.1, todo isto fai que os custos software neste proxecto sexan nulos, a excepción da necesidade de unha subscrición ao servizo *premium* de Spotify. Na táboa 2.1 podemos observar os diferentes prezos do servizo, neste caso o plan elixido é o familiar polo tanto o custo mensual ascende a 2,67 € mensuais e aproximadamente de 14 € durante a realización do proxecto.

Táboa 2.1: Custos dos plans de Spotify

| **Nome** | Individual | Duo | Familiar | Estudantes |
|---|---|---|---|---|
| **Número de contas** | 1 | 2 | 6 | 1 |
| **Prezo mensual** | 9,99 € | 12,99 € | 15,99 € | 4,99 € |
| **Prezo por conta** | 9,99 € | 6,50 € | 2,67 € | 4,99 € |

En canto aos **recursos hardware**, non foi necesario adquirir ningún tipo de equipo, xa que para o desenvolvemento o único necesario é un ordenador con conexión a Internet. Sopesouse a idea de necesitar un servidor na nube para despregar a aplicación o completo, pero debido ao pequeno tamaño da aplicación poderíase recorrer a un servizo gratuíto se fose necesario.

Polo tanto dedúcese que respecto a viabilidade económica este proxecto é completamente factible xa que os custos vense moi reducidos incluso os necesarios son asumibles xa que o servizo *premium* de Spotify é algo que ten mais utilidades que o desenvolvemento.

## 2.2 Viabilidade técnica

A viabilidade técnica é a referente a o funcionamento do sistema, condiciona se as características tecnolóxicas permiten que o proxecto acade un fin exitoso. Neste caso o software escollido para o desenvolvemento aínda que sendo relativamente novo é altamente coñecido e empregado na creación de proxectos similares. Ademais dispóñese de moita documentación e moi completa, incluso titoriais específicos, o que facilita a realización do proxecto polo tanto non presenta unha dificultade técnica alta. Xuntando estes motivos, afírmase que o proxecto é viable tecnicamente.

## 2.3 Viabilidade do mercado

No apartado do estado da arte 3.6 verase que existen diferentes compañías que ofrecen software e hardware comercial que comercializan gramolas virtuais físicas para establecementos onde se pode engadir música directamente dende un aparello, estas compañías atópanse algo implantadas en Norte América e comezan a popularizarse algo en Europa. En canto a software libre non existen opcións comerciais que cumpran este fin dun xeito similar, e o que existe é tedioso de instalar e mais complexo para o administrador. Tan só hai algúns proxectos ou aplicacións móbiles que de forma xeral crean unha gramola virtual de maneira local para que os usuarios poidan engadir cancións, incluso Spotify ten unha función de crear unha sesión para compartir con catro usuarios mais, pero que non é tan doado de utilizar en espazos mais grandes, xa que están plantexados para utilizar en pequenas reunións ou lugares con

poucos usuarios. Non se atopou ningún proxecto que permita crear unha gramola virtual sen manter outra aplicación funcionando e sexa distribuído de forma gratuíta. Por estes motivos pódese afirmar que este proxecto ten cabida no mercado, entón pode ser consideralo coma viable.

Poñendo en conxunto as distintas análises de viabilidade pódese resumir que o proxecto ten cabida no mercado, é viable economicamente e tecnicamente. Polo tanto dedúcese que o proxecto é viable e é posible que teña éxito no mercado.

## 2.4 Análise

Co fin de facilitar a democracia musical nas reunións de amigos, bares, festas e demais, xurde a idea de crear unha aplicación que permita aos usuarios escoller as cancións para escoitar. Esta idea xa existe pero non está implantada e dedúcese que o motivo é a dificultade de uso das aplicacións, se para engadir unha canción, é necesaria facer a descarga da aplicación, rexistrarse, procurar a sala e engadir a canción, é normal que os usuarios non teñan gran interese en realizar estes pasos. Algo similar ocorre cos administradores, cantos menos pasos haxa que dar para que os usuarios poidan utilizar a aplicación, mais probable será a súa adopción.

Por estas razóns decidiuse crear unha aplicación web, xa que evita a descarga, cun rexistro sinxelo e unha interface intuitiva que axude aos usuarios na procura dos seus temas favoritos. Vaise desenvolver un frontend baseado en Ionic no que se amosarán todos os datos relacionados coas cancións que están na cola, o número de votos e a súa orde de reprodución. Tamén se vai realizar un backend en Django Rest Framework que nos permitirá a conexión do frontend coa API de Spotify e así levar a lóxica do proxecto a cabo.

A aplicación contará coas seguintes funcións:

- **Xestión da reprodución de música**

  Esta é a principal función do proxecto, permite xestionar a reprodución de música nun cliente de Spotify. O seu obxectivo é engadir a seguinte canción na lista de reprodución da aplicación a cola de reprodución coa finalidade de que esta sexa reproducida ao rematar a canción que está a soar. Deste xeito non se interrompe de ningunha forma o funcionamento normal do cliente de Spotify xa que pódese seguir facendo modificacións no cliente, engadindo cancións a cola ou cambiando a unha nova, e as seguintes soarán na orde correcta debido a que as accións no cliente prevalecen as da aplicación.

- **Creación dunha sesión**

Unha sesión consta dun conxunto de usuarios que engaden cancións a mesma lista de reprodución. Estas constan dun usuario administrador que é xeralmente tamén o propietario da conta de Spotify, é dicir unha sesión conta tan só con un lugar de reprodución. A aplicación permitirá crear diferentes sesións para diferentes grupos de usuarios e diferentes contas de Spotify. Cada sesión pode interpretarse coma un bar ou lugar de emprazamento dunha gramola física, pero por suposto que pode ser usado en oficinas, reunións particulares e calquera sitio onde a música teña cabida.

- **Creación de usuarios**

  Esta función é a que permite rexistrar usuarios en cada unha das salas, unha vez rexistrados estes poderán procurar e engadir cancións a cola de reprodución, tamén poderán realizar votacións en cada unha das cancións o que lles permitirá que as mais votadas sexan reproducidas antes que as que menos votos teñen. O primeiro usuario rexistrado terá o rol de administrador, polo tanto será o encargado de engadir a conta de Spotify para permitir a reprodución de música.

- **Creación dunha cola de reprodución**

  A creación da cola de reprodución é o que permitirá aos usuarios visualizar cales son as cancións que van soar a continuación. Esta estará ordenada de xeito cronolóxico é dicir, as engadidas con anterioridade serán as primeiras en soar, aínda que esta orde poderase ver modificada debido as votacións dos usuarios o que repercutirá en que as cancións mais votadas serán as primeiras en soar independentemente do intre no que se agregou a canción.

- **Procura de cancións**

  A función de procura é a que permite aos usuarios buscar cancións para engadir a cola de reprodución, amosa unha lista de resultados e o usuario ten que elixir a canción desexada. A procura faise no catálogo de Spotify, polo tanto a dispoñibilidade de uns ou outros temas depende da plataforma musical.

- **Votación de cancións**

  Os usuarios terán a posibilidade de votar as cancións que estean na lista de reprodución, cando unha canción é a que maior número de votos ten, será a próxima en ser engadida a cola, independentemente do resto de factores que ordenan a lista de reprodución, o número de votos de cada canción aparece ao seu carón na lista.

- **Elección de dispositivo**

  Debido a que a plataforma de Spotify é multidispositivo, é probable que se atope mais dun cliente aberto a vez, por isto é necesario que o administrador poida elixir dende

que dispositivo quere que a música sexa reproducida. No caso de que a música xa estea soando non é necesario facer esta elección xa que as cancións serán engadidas a cola do dispositivo que está a funcionar nese intre.

- **Poboar a lista de reprodución**

  A función de poboar a lista de reprodución, permite seleccionar unha lista de reprodución de Spotify, pode ser tanto unha que creemos nos coma calquera outra que sigamos, e vanse introducir as cancións de dita lista na nosa lista de reprodución. Deste xeito se consegue unha lista con cancións que son acorde a unha temática e que os usuarios poderán votar para elixir a orde, tamén pode axudarlles a orientarlles ao elixir as cancións que queiran procurar.

## 2.5 Actores

Os actores nunha aplicación especifican un rol xogado por un usuario ou calquera outro sistema que interactúa co suxeito. No proxecto existen tres tipos de usuarios, usuarios sen rexistrar, usuarios rexistrados e administrador.

- **Usuario sen rexistrar**

  O usuario sen rexistrar é o usuario que accede a aplicación web pero non está autenticado. Non se contempla a posibilidade de que os usuarios sen rexistrar poidan consultar o estado de sesións nin realizar ningunha acción sen se rexistrar primeiro. As funcións que poden realizar os usuarios sen rexistrar son, creación dunha nova sesión, rexistrarse nunha sesión e conectarse ao seu usuario previamente rexistrado.

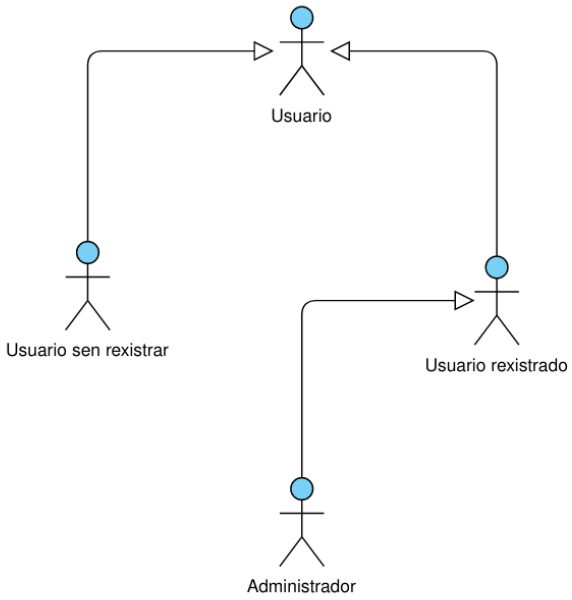
- **Usuario rexistrado**

  O usuario rexistrado é aquel que está autenticado na aplicación web, cada usuario estará asociado a unha sesión e dito usuario poderá realizar accións sobre unicamente esa sesión. As accións que poden realizar son, a visualización da cola de reprodución, buscar cancións no catálogo de Spotify, engadir cancións a cola de reprodución e votar as cancións que xa pertencen a lista de reprodución da sesión.

- **Usuario administrador**

  O usuario administrador é o primeiro usuario rexistrado en cada unha das sesións, pode realizar as mesmas accións que os usuarios rexistrados e ademais é o que interactúa coa conta de Spotify. As accións exclusivas do usuario administrador son, conectar a sesión con unha conta de Spotify, seleccionar o dispositivo no que vai traballar a aplicación e tamén ten a capacidade de poboar a lista de reprodución da aplicación seleccionando unha das listas da conta de Spotify.

Figura 2.1: Diagrama de actores



## 2.6 Requisitos da aplicación

Os requisitos son as necesidades físicas ou funcionais que o produto trata de satisfacer. Os requisitos poden ser funcionais ou non funcionais. Os requisitos non funcionais son aqueles que non están especificamente relacionados coa funcionalidade do sistema. Os requisitos funcionais son aqueles que describen o comportamento ou función particular dun sistema ou software cando se cumpren certas condicións.

### 2.6.1 Requisitos funcionais

- Amosar información da cola de reprodución.
- Amosar os resultados de procura.
- Permitir engadir cancións a cola.
- Posibilitar a creación de sesións.
- Permitir facer unha procura de cancións.
- Permitir o rexistro de usuarios.
- Permitir a autenticación dos usuarios.
- Permitir facer votacións individualmente nas cancións.
- Permitir a engadir unha lista completa a lista de reprodución.

### 2.6.2 Requisitos non funcionais

- **Deseño:** minimalista fácil de comprender e sinxela de usar.
- **Idioma:** a aplicación estará en inglés xa que a maior parte dos usuarios de Spotify é en Norte América e Europa, polo tanto un gran volume comprenderá o idioma sen problema.
- **Consistencia:** o modelo da aplicación mantense actualizado sen perder datos.
- **Seguridade:** tanto as comunicacións coa API coma o almaceamento de credenciais debe ser seguro.
- **Fiabilidade:** Os erros en produción están controlados para poder dar unha resposta ó usuario en caso de que se dean.

Capítulo 3

# Tecnoloxías

Neste terceiro capítulo explicase as tecnoloxías utilizadas durante o proceso de desenvolvemento, as linguaxes usadas no proxecto xunto coas librarías, os frameworks e utilidades externas, por último explicarase o estado da arte respecto a nosa idea.

## 3.1 Ferramentas de desenvolvemento

Para a creación deste proxecto utilizouse unha estrutura de desenvolvemento sinxela realizada na maior parte nun terminal Linux xa que este é moi versátil e lixeiro, a continuación describense as ferramentas que foron utilizadas e comentando as súas características principais.

Comezando polo sistema operativo, utilizouse **Linux** todo o código do proxecto foi desenvolvido en máquinas con *Arch Linux*, unha distribución moi lixeira acompañada dunha gran cantidade de aplicacións instalables diretametne co seu xestor de paquetes *Pacman*, tamén é unha das primeiras en recibir actualizacións de paquetes coma o *Kernel*, ademais conta con unha enorme comunidade que permite documentarse e resolver problemas rapidamente.

Para e edición do código escolleuse **NeoVim** trátase un editor de texto na terminal, creado para aumentar a eficiencia ao editar calquera tipo de texto. É un fork de *VIM*, e está deseñado do facilitar o seu mantemento, a súa extensibilidade e usabilidade, alenta aos seus usuarios engadiren contribucións. Aínda que non se trata dun Integrate Development Environment (IDE) grazas a comunidade existen múltiples complementos que fan o seu uso moi similar a un destes, pero sendo moito mais lixeiro en canto a recursos utilizados. Foi utilizado para o desenvolvemento do código e da memoria, aquí pódese atopar a configuración utilizada aínda que permanece en constante cambio.

Coma sistema de control de versións utilízase **Git** é un sistema distribuido de código aberto, deseñado para manexar todo tipo de proxectos, caracterízase principalmente pola súa velocidade, rendemento, usabilidade e flexibilidade. Facilita de gran maneira o rexistro de cambios

que se fan no código o que tamén permite o desenvolvemento de código de forma colaborativa.

Para o aloxamento do proxecto, facilitando así a súa compartición e a copia de seguridade utilizouse **GitHub**, que é unha plataforma de desenvolvemento colectivo que utiliza o sistema de control de versións de *Git*, utilizase principalmente para o control de versións e o almacenamento de código fonte.

No apartado de probas utilízase **Firefox developer edition** para comprobar o funcionamento da aplicación. Trátase dun dos navegadores mais utilizados, é de código libre e conta con unha comunidade que desenvolve unha gran cantidade de complementos. Nesta versión en concreto ponse énfase en facilitar o traballo aos desenvolvedores, xa que conta con características coma a visualización de cuadrículas CSS ou un depurador para *JavaScript*. Polo tanto pensouse que é o mais axeitado para o desenvolvemento de gran parte do frontend.

Para comprobar a compatibilidade tamén usáronse diferentes navegadores, **Firefox** na súa versión normal e na versión móbil, do mesmo modo **Google Chrome** probouse en ambas versións, na móbil e na de escritorio. Nos navegadores específicos dos sistemas *Windows* máis *OS X*, que son *Edge* e *Safari* respectivamente tamén foron utilizados mais sen facer probas exhaustivas.

No seu comezo era tan só unha extensión para o navegador, pero **Postman** agora é unha completa plataforma que permite a creación o uso de APIs de maneira mais sinxela, simplificando todos os pasos na creación destas, o que permite o seu desenvolvemento dunha forma máis rápida e mellor. No proxecto a utilizamos en lugar de HTTPie [1] xa que se coñecía o seu funcionamento con anterioridade.

Para a realización dos mockups do frontend, utilizamos **Figma** un editor de gráficos vectoriais, tamén e unha ferramenta de xeración de prototipos baseada na web, aínda que conta con aplicacións para o escritorio.

Os diagramas que podense ver ao longo da memoria foron realizados coa ferramenta online **Visual paradigm**, trátase dunha ferramenta para a creación de Linguaxe unificado de modelado (UML), infografías, gráficas interactivas ...Contén unha gran cantidade de imaxes prediseñadas que axudan a rápida produción de ditos elementos visuais.

Coa idea de controlar o tempo invertido e a xestión de tarefas utilizouse **Superproductivity** que é unha ferramenta de código libre deseñada precisamente para isto, permite levar un rexistro das tarefas e os tempos invertidos para cada unha de elas, ademais de contar con integración coas principais plataformas de desenvolvemento colectivo coma *Jira* ou *GitHub* o que permite a asignación de tarefas ou ¨tickets¨ de forma doada. Por desgraza non se utilizou durante todo o proxecto e tan só ao final polo que non foi posible afinar ao máximo os tempos empregados en total.

## 3.2 Frameworks e librarías

No desenvolvemento do proxecto utilizamos principalmente dous frameworks para o frontend e o backend e unha libraría que nos facilita as peticións a API de Spotify, a continuación describimolos.

Para o desenvolvemento do frontend empregamos **Ionic** un framework de código aberto que engloba un conxunto de ferramentas para o desenvolvemento de Progressive web application (PWA)[1], enfócase na experiencia do usuario e no interface, proporcionando múltiples controles para xestionar as iteracións. Unha das súas características mais chamativas é que permite co mesmo código a creación de aplicacións web, e aplicacións para os sistemas mais utilizados hoxe en día, *iOS* e *Android*. Escollimos esta plataforma debido a súa integración e a facilidade para desenvolver aplicacións web.

*Ionic* permite a integración cos frameworks *React*, *Vue* e *Angular* decantámonos por este último. **Angular** é un framework baseado en compoñentes para a xeración de páxinas web escalables, está desenvolvido en *TypeScript* e *HTML*, utiliza unha ampla gama de librarías que integran características coma as redirecións ou manexo de formularios. Decantámonos por este framework xa que é o que mais tempo leva funcionando en *Ionic* e contén características coma os observables que facilitan a xestión das peticións ao backend.

Na parte do backend eliximos **django**, trátase dun framework de código aberto escrito en *Python*. Na súa orixe foi creado para xestionar un xornal de noticias, facilita a creación de sitios webs complexos, pon énfase na reutilización da conectividade e a extensibilidade dos compoñentes e no desenvolvemento rápido. No noso caso utilizamos *Django Rest Framework*, que contén características mais específicas para a creación de APIs RESTful o que permite a transferencia de información entre as interfaces e as bases de datos, precisamente é o axeitado para o noso backend.

Por último, referente as librerías utilizamos **Spotipy** é unha libraría de Python que permite a execución das consultas na API de Spotify. Ademais tamén inclúe a xestión de *tokens* para gardar as credenciais de acceso dos usuarios, polo tanto dunha forma doada podemos interactuar coa plataforma de música. Tivemos en conta outras librerías coma Tekore pero a descartamos por non contar con tanta documentación e non satisfacer as necesidades do mesmo xeito.

## 3.3 Linguaxes

Sendo este un proxecto que utiliza un backend e un frontend con varios frameworks foi necesaria a utilización de varias linguaxes de programación.

[1] Aplicacións, web progresivas

Na parte do frontend para a visualización da aplicación utilizamos **HyperText Markup Language (HTML)** que é unha linguaxe de marcado, trátase dun estándar para documentos que se desexen ver nun navegador, utilízase na maioría das páxinas webs xa que define a estrutura básica destas. En conxunto con outras tecnoloxías coma Cascading Style Sheets (CSS) permite cambiar a aparencia ou con JavaScript podemos cambiar a súa funcionalidades crear páxinas web dinámicas.

Para editar o aspecto das páxinas web utilizamos un procesador de CSS chamado **Sass**. É unha linguaxe de *scripting* que permítenos xerar de forma automática follas de estilo, engadindo características que non contén CSS e que son propias de linguaxes de programación coma poden ser as variables, funcións, selectores aniñados, herdanza... Isto axúdanos a manter unha organización modular dos estilos, ademais de que reduce o tempo de creación e mantemento do CSS.

Coma explicábamos con anterioridade, **JavaScript** serve para engadir melloras no interface e dinamizar páxinas web. É unha linguaxe de programación dinámica de scripting, orientada a obxectos, permite engadir contido dinámico e interactivo as páxinas web. Normalmente executase no propio navegador e pode ser inxerido en calquera parte do código HTML. Aínda que é amplamente utilizado na web non é exclusivo, utilízase tamén no desenvolvemento de xogos ou por exemplo *widgets*.

Debido a que *JavaScript* non pode detectar erros, utilizamos **TypeScript** que é un superconxundo de *JavaScript* deseñado para aplicacións extensas que a través dun compilador traducese a *JavaScript*, engade tipos estáticos e obxectos baseados en clases, principalmente é usado para o desenvolvemento de aplicacións do lado do cliente.

Na parte do backend utilizamos tan só **Python** xa que Django está escrito nesta linguaxe e todas as funcións que realizamos tamén. Trátase dunha linguaxe de programación interpretado, que fai fincapé na lexibilidade do código e utiliza a indentación para isto, do mesmo xeito tamén aposta pola modularidade e a reutilización do código, conta con un amplo repositorio de paquetes que estenden a súa funcionalidade de gran maneira. É unha linguaxe de alto nivel e multiparadigma xa que acepta tanto programación orientada a obxectos, imperativa e funcional. Tamén é un das linguaxes maior valuadas polos desenvolvedores.

Para o manexo da base de datos *Django* utiliza **SQL** que é unha linguaxe cun dominio específico, é usado para a administración, recuperación de bases de datos relacionais, particularmente útil usando modelos de datos. Unha das súas principais características é que usa álxebra relacional para efectuar as consultas, co fin de recuperar de forma sinxela información das bases de datos e para realizar cambios nela.

Por último, para a realización da memoria utilizamos **LaTeX**, é un sistema de composición de textos de alta calidade. En xeral é usado para crear artigos e documentos técnicos e científicos aínda que pode utilizarse para todo tipo de textos. Segue a filosofía de separar a

presentación e o contido, deste xeito os autores poden enfocarse no contido sen atender de forma simultanea a parte visual. Cando se prepara un documento o escritor especifica unha estrutura lóxica utilizando conceptos simples coma capítulo, sección ou táboa, e LaTeX encargase do formato, de todos modos permite a edición do estilo aínda que sexa dunha forma separada.

## 3.4 Sistema de xestión de base de datos

Xa sabemos que *Django* utiliza *SQL* para o manexo das bases de datos, pero é necesaria una implementación dun sistema xestor, no noso caso **SQLite**. É unha libraría escrita en *C* que implementa un motor de bases de datos SQL, lixeiro, rápido, autocontido e cunha implementación completa para SQL. É ideal para aplicacións que requiren unha instalación pequena e nas que a maioría das peticións son de tan só lectura. No noso proxecto o usamos para almacenar tanto os usuarios, as sesións e as cancións.

Debido a que *SQLite* está deseñado para bases de datos lixeiras non ten un gran soporte de concorrencia. O que quere dicir que non sempre vai ser capaz de xestionar os interbloqueos entre diferentes fíos de proceso, polo tanto é posible que teñamos erros de bloqueo da base de datos e non sexa posible acceder os datos requiridos en canto o número de usuarios da plataforma sexa moi grande. Por sorte Django permite cambiar o xestor de bases de datos e isto poderíase facer se comezásemos a sufrir estes problemas debido a alta demanda de usuarios e poderíamos poñer en funcionamento outro motor coma podería ser *mySQL* ou *MariaDB* xa que estes manexan de mellor xeito os bloqueos.

## 3.5 APIs

A base deste proxecto é reproducir música e para isto necesitábamos obter a música dende algún lugar. Nos primeiros momentos ponderamos o uso de varias plataformas de música conxuntamente coma poderían ser *YouTube*, *Deezer* ou *Apple Music* pero finalmente decantámonos por reproducir a música directamente na plataforma e facer modificacións na súa cola de reprodución o que facilitaría o funcionamento para os usuarios, para isto teríamos que escoller tan só unha. Escollemos *Spotify* xa que conta con una enorme biblioteca musical e tamén é unha plataforma que está moi afianzada, moitos locais tan só utilizan este servizo para toda a música que reproducen. Outra baza ao seu favor é a súa completa API que permítenos xestionar case todas as funcionalidades que poderíamos utilizar nun dos seus clientes, coma son a procura ou a xestión de listas de reprodución aínda que notamos en falta algunha funcionalidade coma é a consulta da cola de reprodución, xa que esta API tan só permite engadir cancións a esta e ningunha outra acción o que podería ternos facilitado o traballo. Por

último tamén nos axudou a escoller esta plataforma a súa ampla documentación específica para a API que é moi detallada e conta con exemplos e incluso un cliente para facer probas.

Aínda que poderíase utilizar a **API de Spotify** directamente, nos utilizamos unha librería de *Python* chamada *Spotipy*, xa que implementa todas as funcionalidades da API e ademais facilita a obtención e gardado dos *tokens* dos usuarios, contando tamén con unha documentación extensa explicando cada unha das súas funcións.

## 3.6 Estado da arte

Na actualidade existen diferentes tipos de gramolas, en xeral todas son dixitais e obteñen a información a través de Internet. Hai compañías que ofertan o servizo de obter un aparello físico para dispoñer nun local, e que alí os clientes poidan dun xeito similar a antigamente facer a súa elección musical. Do mesmo xeito existen outras solucións que están baseadas en aplicacións, dun xeito similar ao noso proxecto, os clientes poden conectarse a estación do lugar onde están e alí engadir cancións para a súa reprodución. Normalmente estas plataformas contan co seu propio reprodutor de música e o seu propio catálogo, xeralmente bastante extenso pero non tanto coma calquera servizo de música xa que contan coas cancións mais actuais e os clásicos. Non existe un mercado moi amplo aínda que destacan algunhas aplicacións que se están comezando a implantar tanto nos Estados Unidos coma en Europa.

Comezando polas gramolas físicas temos a compañía **TouchTunes** está expandida por Norte América e parte de Europa dispoñible en 75.000 bares é a mais coñecida con diferenza, dispón dun gran catálogo de música, sistema de adaptación da música ao local e diferentes ferramentas para o entretemento dos clientes. Dispoñen de 3 tipos de gramolas, dúas de parede, unha que tan só sirve de reprodutor 3.1a e outra que contén mais funcionalidades coma o karaoke, unha última adaptable para axeitala o estilo do local coma por exemplo nun barril 3.1b. Todas funcionan con unha gran pantalla táctil, para facer a selección aínda que conta con unha aplicación móbil para este mesmo fin. Utiliza un sistema de créditos que se poden consumir nos diferentes establecementos nos que está implantada a plataforma, polo tanto o seu uso non é gratuíto. Páxina oficial TouchTunes

Como directa competidora de TouchTunes atopamos **AMI Entertainment**, comezou creando pianos nos que introducindo unha moeda podíase elixir algunha das cancións, posteriormente dedicaronse a facer gramolas físicas e nos nosos días seguen coa dixitalización destas. Conta con uns 30.000 aparellos repartidos por todo o mundo. Ao igual que TouchTunes, tamén con 3 tipos de gramolas diferentes, moi similares as da súa máxima competidora pero inclúen unha de pé 3.2a, similar a unha das gramolas antigas. Tamén conta con aplicación móbil para engadir cancións, e tamén funciona cun sistema de créditos pra facer soar a música que mais che gusta. Páxina oficial de AMI entertainment

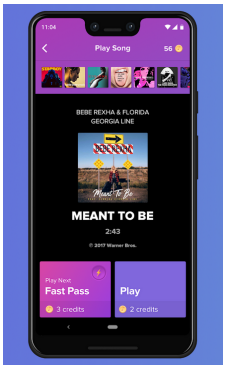
Figura 3.1: TouchTunes




(a) TouchTunes Virtuo




(b) TouchTunes Fusion




(c) Aplicación móbil

Figura 3.2: Ami Entertainment



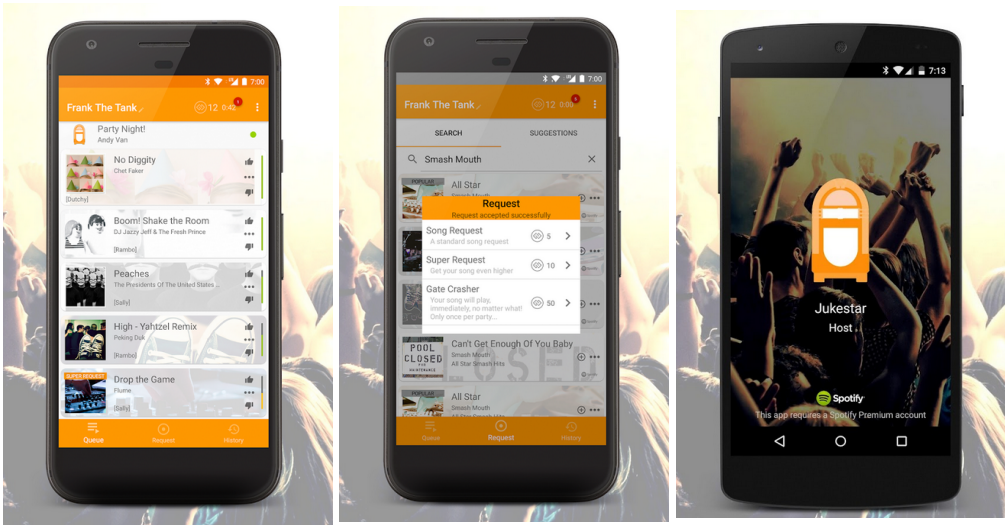
(a) NGX Infinity




(b) Aplicación móbil

No apartado de aplicacións móbil que teñan unha funcionalidade similar a unha gramola atopamos **Jukestar**, consiste en dúas aplicacións unha anfitrioa 3.3c e outra para os invitados 3.3a, o anfitrión ten que conectar a súa conta de Spotify na aplicación e xerar unha festa despois compartir a festa cos seus invitados, estes poden incluír cancións ou votar polas da súa preferencia, estas características funcionan cun sistema de créditos de pago 3.3b. É unha idea similar a da nosa aplicación, xa que utiliza Spotify para a reprodución de música pero necesita un cliente para o anfitrión e outro para os invitados o que complica o seu uso. Jukestar

Figura 3.3: Jukestar App




(a) Jukestar gramola (b) Solicitude de nova canción (c) Aplicación anfitrión

Outra aplicación similar é **Outloud** permite a reprodución de música dende Spotify, SoundCloud e incluso Apple Music para os trebellos compatibles. A idea e similar, créase unha playlist que faría de sesión e os usuarios poden conectarse xa sexa dende un enlace de invitación ou buscando por lugares próximos, despois poden engadir cancións e votar as que son mais do seu agrado, tamén permite o acceso dende a web sen necesidade de instalar a aplicación. Unha característica que inclúe e non existe noutras aplicacións é a posibilidade de proporcionar información da sesión nunha pantalla central 3.4b, nesta inclúense as seguintes cancións, as cancións recén engadidas e incluso quen son os usuarios que reciben mais votos nas súas cancións. Esta aplicación está pensada para usala en calquera lugar: vodas, ximnasios, lugar de traballo, festas particulares, reunións con amigos ... e pódese descargar en dispositivos Android ou iOS. Trátase dunha aplicación sinxela ademais de ser gratuíta, o único problema que podemos ver é que a aplicación necesita estar funcionando no dispositivo no que soa a música. Páxina oficial Outloud

Por outra banda, cambiando as aplicacións por servizos baseados na web, os que non necesitan aplicacións, existe **Jukebox Today** tratase dun servizo web que permite crear sesións e

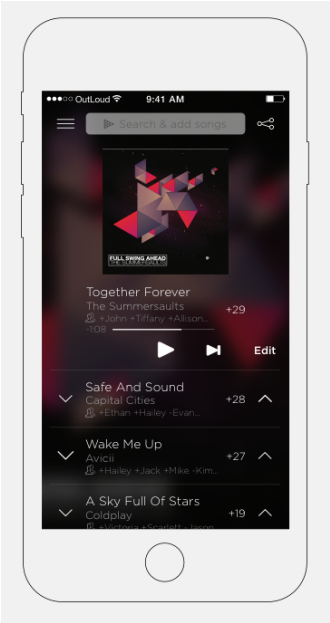
compartila cos usuarios. Permite que a música se reproduza de forma sincronizada en varios dispositivos ou controlar a reprodución dende calquera dos que estea activo, incluso conta con un sala de conversación, para os usuarios. Obtén a música dende YouTube polo tanto o seu catálogo é tan amplo como o da plataforma. Aínda que podería funcionar coma unha gramola non é a súa idea principal, xa que todos os usuarios poden facer modificacións na lista de reprodución ou poñer a pausa, trátase dun servizo gratuíto. Jukebox Today

Con funcionamento completamente diferente **Raveberry** é un proxecto no que se instala nunha RaspberryPi un servidor musical multiusuario, é dicir, unha gramola. Contén unha interface intuitiva para a solicitude de cancións e cambiar a orde de acordo coa puntuación que os usuarios fan. Soporta as plataformas de Youtube, Spotify e SondCloud, incluso permite engadir arquivos locais. Está baseado en Python e contida no repositorio *PIP* e dispoñible tamén en *Docker* polo tanto é instalable en múltiples dispositivos aínda que é nunha RasberryPi onde se poden desfrutar a maioría das súas características que son moitas e moi ben pensadas. Citamos as mais destacadas que converten a Raveberry nun gran proxecto:

- **Streaming remoto:** permite reproducir música sincronizada noutros dispositivos.
- **Bluetooth:** pódese conectar a RaspberryPi a un altofalante bluetooth que permite evitar os cables.
- **Hotspot:** creación dun punto WiFi no que os usuarios pódense conectar para usar a aplicación ou acceso a internet.
- **Roles de usuario:** permite engadir permisos a certos usuarios, por exemplo o control da reprodución.
- **Interface gráfica:** o administrador pode elixir a configuración desexada nunha cómoda interface.
- **Shareberry:** é unha aplicación complementaria que permite engadir cancións directamente dende o teléfono móbil.
- **Visualización de pantalla:** usando a aplicación cava, pódense ver as frecuencias de música nunha pantalla conectada 3.6a.
- **Visualización de audio:** pódese engadir dispositivos LED e Raveberry fará que funcionen ao ritmo da música.

Para o funcionamento de algunhas destas características é necesaria a inclusión de algún hardware adicional para o que o proxecto xa está preparado, por exemplo unha antena WiFi para a compartición da conexión, a integración dunha tarxeta de son para a mellora da calidade do son ou as luces LED que son controladas polos protocolos *i2c* e *spi*. Contén ademais,

Figura 3.4: Outloud App

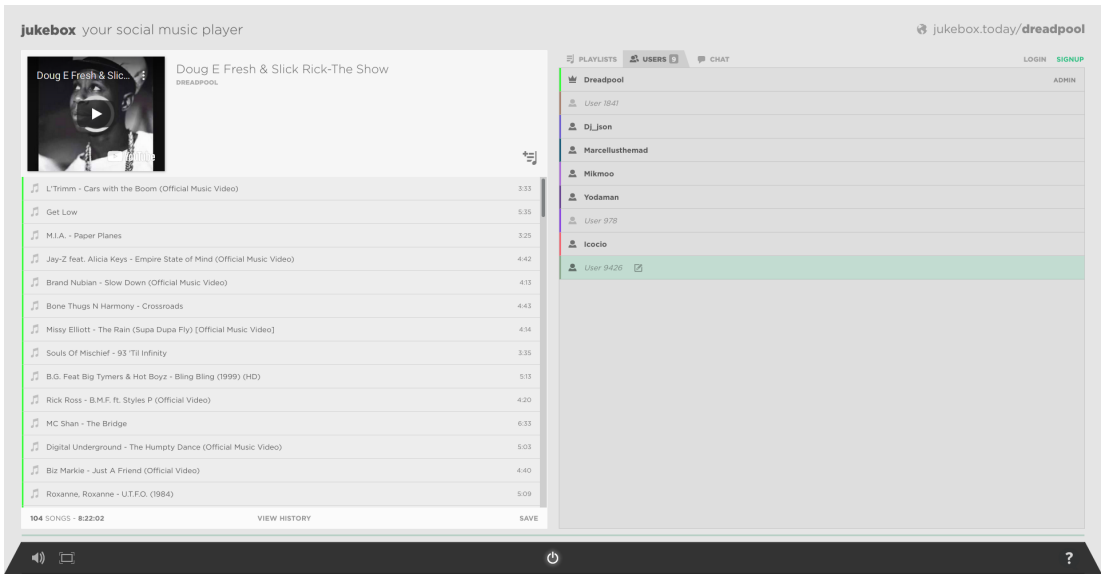



(a) Aplicación de Outloud



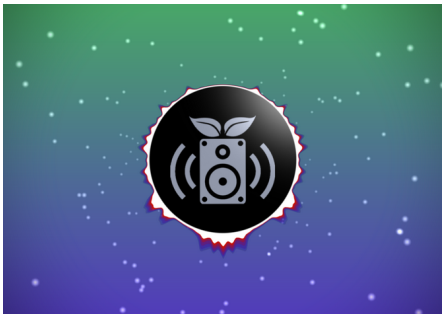

(b) Modo pantalla central

Figura 3.5: Captura dunha sala en Jukebox Today

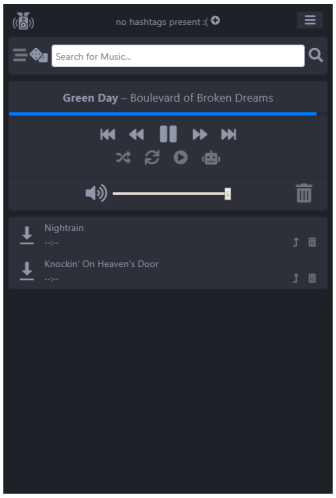


unha ampla documentación que fai que a súa instalación básica seña moi sinxela e trátase dun proxecto de código aberto polo tanto gratuíto. Repositorio de Raveberry en GitHub

Figura 3.6: Raveberry Server



(a) Visualización de pantalla




(b) Interface

Estes son os proxectos que atopamos cun funcionamento similar ao noso traballo, ningún destes cumpre todos os nosos requisitos aínda que varios aproximase a ideas similares, por exemplo *Outloud* necesita unha aplicación aberta polo tanto no caso de esta estar pechada o servizo deixaría de funcionar, desta forma se dificulta o uso para o administrador. Destes traballos obtivemos ideas que poden ser implementadas no noso proxecto e poden engadir funcionalidades interesantes para o usuario final.

Capítulo 4

# Planificación

PARA a planificación dos proxectos é necesaria unha metodoloxía, xeralmente estas seguen unha serie de pasos para o desenvolvemento, neste capítulo explicarase a metodoloxía utilizada e os pasos do proceso de desenvolvemento.

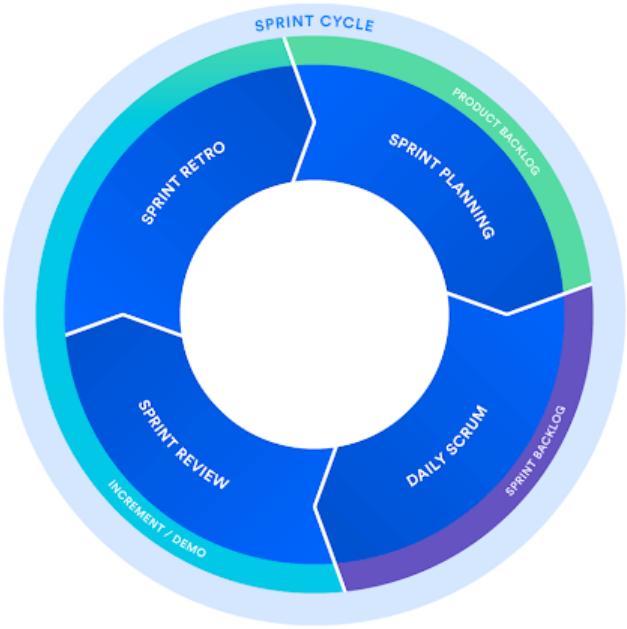
## 4.1 Metodoloxía

Durante o desenvolvemento deste proxecto seguiuse a metodoloxía de Scrum 4.2, aínda que tivemos que adaptala ao noso modo de traballo debido a que está pensada para o desenvolvemento en grupo e non de forma individual como foi o caso neste proxecto. De todas formas é unha metodoloxía perfectamente aplicable xa que basease no aprendizaxe continuo, polo tanto asúmese que o equipo non ten todos os coñecementos ao principio do proxecto e vai ir evolucionando cando se vaia acadando mais experiencia.

## 4.2 Scrum

Scrum é un proceso de desenvolvemento áxil. Permite un desenvolvemento rápido e por iteracións, permite inspeccionar o software creado e cambiar as preferencias de traballo no lugar dunha planificación e execución completa dende o principio. É un método no que se aplican de maneira regular un conxunto de boas prácticas para traballar de forma colaborativa e obter o mellor resultado posible. Os equipos organizasen entre eles para discutir as funcionalidades e a prioridade destas, deste xeito despois de cada iteración se pode facer unha demostración do produto nese momento. Na figura 4.1 pódese ver como é un ciclo dun sprint en Scrum.

Figura 4.1: Ciclo de sprint en Scrum




### 4.2.1 Artefactos Scrum

Os artefactos son ferramentas que se fabrican para solucionar un problema, en scrum existen tres artefactos que o equipo irá revisando en cada ciclo.

- **Product Backlog:** é a lista principal de traballo que debe realizar o propietario do produto, trátase dunha lista dinámica de funcións, requisitos, melloras e correccións que actúa coma entrada para o sprint backlog. Basicamente é unha lista de tarefas por facer do equipo, estas pódense cambiar de prioridade e se están revisando constantemente xa que é posible que algunhas tarefas deixen de ser necesarias ou os problemas se solucionen de forma diferente, trátase dunha lista dinámica que vai cambiando cos diferentes sprints.

- **Sprint Backlog:** trátase da lista de elementos, historias de usuario ou correccións de erros seleccionados polo equipo de desenvolvemente para súa implementación no ciclo actual do sprint. Antes de cada sprint, na reunión de planificación, o equipo elixe os elementos nos que vai traballar durante ese sprint. O sprint backlog pode ser flexible e evolucionar durante o sprint.

- **Incremento:** é o obxectivo do sprint, o produto final utilizable ao finalizar este. Depen-

dendo do equipo a finalización do sprint pode ser diferente, cando se lanza un produto ao cliente, se finaliza unha parte dunha versión mais grande ...Ao rematar o incremento volve a comezar o ciclo e comeza outro sprint ata que o produto estea finalizado.

### 4.2.2 Protocolos ou eventos de scrum

Os equipos de scrum realizan periodicamente eventos secuenciais ou reunións. É a parte mais variable xa que dependendo do equipo elimínanse algúns destes eventos, a continuación describiremos todos os protocolos na orde que se executan normalmente.

- **Organización do backlog:** tamén coñecido coma limpeza do backlog, é responsabilidade do propietario do produto, esta persoa encárgase de facer o mantemento da lista do backlog en base aos comentarios dos usuarios e ao equipo de desenvolvemento.

- **Sprint Planning:** esta reunión é dirixida polo scrum master, todo o quipo de desenvolvemento planifica o traballo que se vai realizar durante o sprint actual e decide o obxectivo do sprint. Tamén se fai unha estimación da súa duración e se obtén o Sprint Backlog.

- **Sprint:** é o período de tempo real que o equipo de scrum traballa de forma conxunta para finalizar un incremento. A duración dun sprint xeralmente é de dúas semanas ou un mes. Canto mais complexo, mais incógnitas haxa, mais corto debería de ser o sprint.

- **Daily Scrum:** trátase dunha reunión diaria duns 15 minutos de duración. O obxectivo é que todos os participantes do equipo estean en sintonía e se adecúen ao obxectivo do sprint cun plan para as seguintes 24 horas. É habitual comezar facendo que cada un dos membros do equipo respondan tres preguntas ¨Qué fixeches onte?¨, ¨Que vas facer hoxe?¨, ¨Que problemas atopaches?¨, é dirixida polo Scrum Master.

- **Sprint Review:** ao final do sprint, o quipo reúnese nunha sesión informal para ver unha demostración ou inspeccionar o incremento. O equipo de desenvolvemento amosa os elementos do backlog que agora están rematados as partes interesadas e aos compañeiros do equipo, deste xeito poden recibir comentarios ou suxestións.

- **Sprint Retrospective:** o equipo reúnese para documentar e analizar que foi o que funcionou e que non durante o sprint, nun proxecto, nos protocolos etc. A idea é crear un lugar onde o equipo poda centrarse no que saíu ben e que pode mellorarse para a próxima vez.

### 4.2.3 Roles

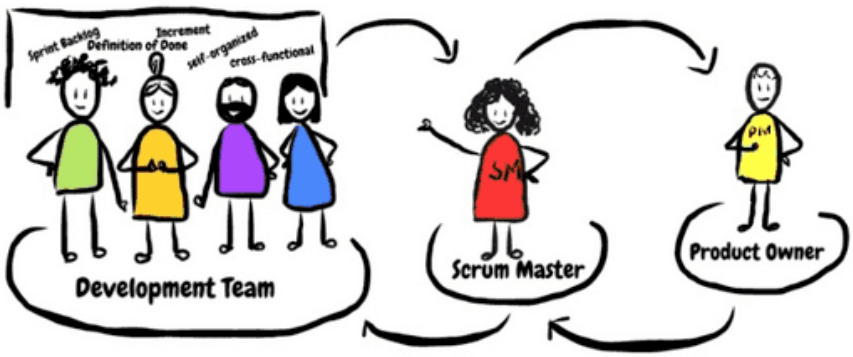
Nesta metodoloxía existen diferentes roles que interpretan distintas persoas, son os seguintes:

- **Product Owner:** son os que mellor coñecen o produto. Están centrados en entender os requisitos empresariais dos clientes e do mercado para despois dar prioridade o traballo que o equipo de enxeñería debe realizara para cumprilos. Crean e xestionan o backlog do produto, asocianse co negocio e co equipo para asegurarse de que todo o mundo entende o traballo, achegan ao equipo directrices claras sobre as funcionalidades que entregar a continuación, tamén deciden cando lanzar o produto. O seu obxectivo é que o produto maximice o seu valor.

- **Scrum Master:** os expertos en scrum son os maiores especialistas en scrum do equipo. Proporcionan formación aos equipos, aos propietarios de produto e a empresa no proceso. Coñecen profundamente o traballo que realiza o equipo e pode axudalo a optimizar a súa transparencia e o fluxo de entrega. Planifican os recursos necesarios para organizar os prazos dos sprints, as reunións rápidas a revisión de sprints e as retrospectivas.

- **Equipo de desenvolvemento:** son os encargados de sacar o traballo adiante. Coñecen as practicas de desenvolvemento sostible. Os equipos mais eficaces teñen unha relación estreita, atópanse na mesma localización e están compostos por entre cinco e sete membros. Os membros do equipo teñen diferentes habilidades, fórmanse entre eles para que ningún quede atrasado. Impulsan o plan en cada sprint, prevén canto traballo creen que poden finalizar ao longo da iteración.

Figura 4.2: Roles en Scrum



### 4.2.4 Scrum no proxecto

Debido as características do proxecto, algúns dos aspectos da metodoloxía foron adaptados para adecuarse ao proxeto. Scrum contempla esta posibilidade é normal que os equipos adapten o método de traballo as súas necesidades, deste xeito séguese a traballar coa mesma metodoloxía e se obteñen as súas vantaxes sen os inconvenientes que non se poidan subsanar.

Ao realizarse de maneira individual e non dipoñer dun equipo, os roles de Scrum Master e o Equipo de desenvolvemento foron realizados polo alumno, mentres que o de Product Owner polo profesor.

Cada Sprint Planning tivo unha duración de entre dúas semanas e catro, o Daily Scrum foi substituído por un repaso das tarefas realizadas e un diario de traballo e o Sprint Planning realizado de forma individual.

Para a creación do Product Backlog se realizaron reunións ao principio do desenvolvemento para a concreción dos aspectos fundamentais da aplicación, este foi adaptándose co paso do tempo segundo as versións funcionais do produto aproximandose cada vez mais aos requisitos finais como indica scrum.

Os Sprint Backlog foron seleccionados polo alumno segundo as prioridades establecidas inicialmente, tratouse de seguir unha orde lóxica de desenvolvemento de funcionalidades, primeiro as mais básicas que dan xeito ao proxecto e posteriormente as mais específicas que son as que o definen finalmente.

As reunións do Sprint Review coma o Sprint Retrospective foron reducidos debido as características individuais do proxecto.

## 4.3 Planificación inicial

Unha vez realizado a análise (capítulo 2) se decidiu dividir o traballo en areas, axeitando estas a planificación da metodoloxía:

- **Configuración inicial do proxecto:** Esta primeira fase incluíu a recolección de información sobre as ferramentas a utilizar para a posta en marcha do proxecto, o estudo de diferentes métodos para a implementación das bases do proxeto e a configuración do entorno de desenvolvemento, tamén se fixo a inicialización dos frameworks.

- **Desenvolvemento do backend:** Posteriormente, creouse a lóxica do backend, desenvolveuse a API propia, crearonse as peticións necesarias da aplicación ao API de Spotify e por último a configuración da xestión de usuarios no backend.

- **Creación da base de datos:** Na terceira fase, fíxose o deseño da base de datos, modificouse o modelo de usuarios de Django e se crearon o resto dos modelos. Tamén se

estudaron as posibilidades de almacenamento de datos na parte do frontend.

- **Xeración do espazo web:** Antes da implementación do frontend fíxose un deseño do entorno web en mockups 4.3, posteriormente se trasladou este deseño ao espazo web creando as diferentes páxinas.

- **Conexión do frontend co backend:** Por último conectouse completamente o backend co frontend, implementaronse as vistas e controladores do frontend, creouse a conexión con JSON web Token (JWT) para a securización dos datos e completouse a administración de usuarios.

## 4.4 Adaptación da planificación a Scrum

O desenvolvemento do proxecto se dividiu en 5 fases ou sprints, as cales tiveron unha duración de entre 3 e 5 semanas dependendo da complexidade, a metodoloxía foi adaptada a natureza do proxeto. Se realizou un Sprint Planning 4.2.2 ao principio de cada fase onde se obtivo o Sprint Backlog 4.2.1 e ao finalizar cada unha se fixo o Sprint Review 4.2.2 para a creación do seguinte Sprint Backlog. Durante a realización do proxecto, tamén se levou un diario de traballo, no que se levaba rexistro das tarefas realizadas, isto tamén facilitou a organización.

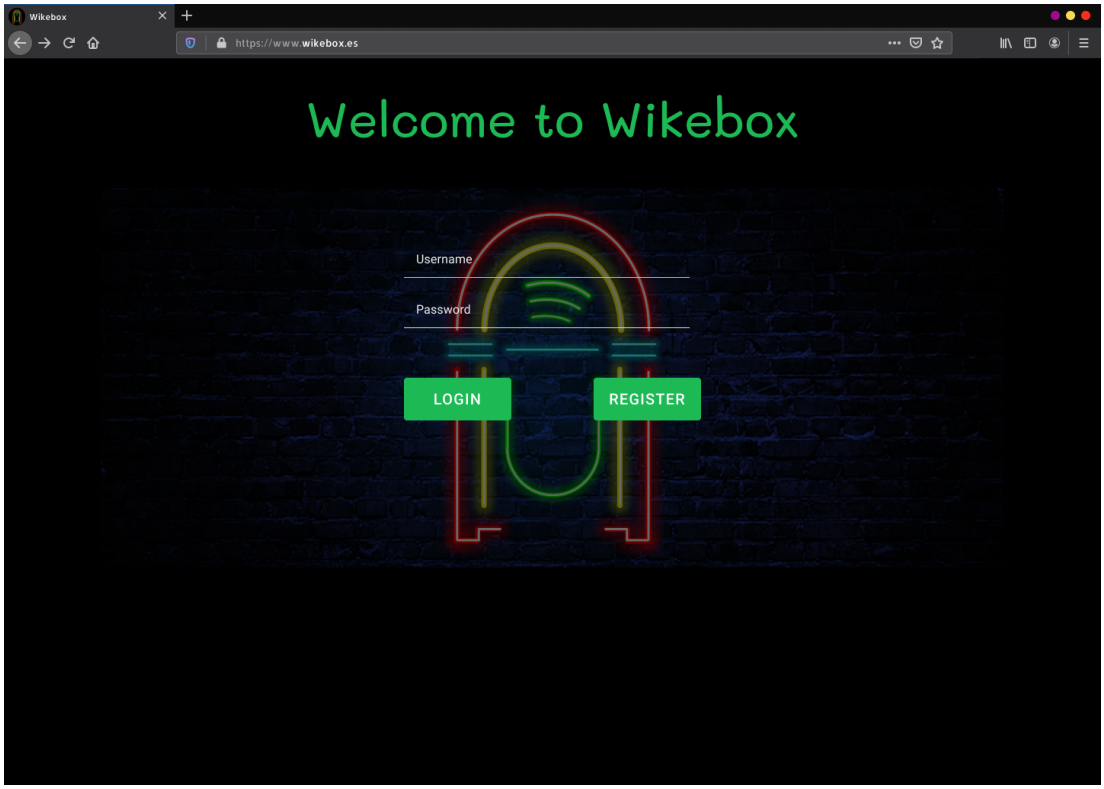
### 4.4.1 Primeiro sprint

No primeiro sprint levouse a cabo a implementación da primeira fase, creouse o Product Backlog 4.2.1 baseado nos requisitos funcionais. Posteriormente estableceuse o espazo de traballo, configurouse o editor NeoVim cunha selección de plugins adaptados aos frameworks elixidos e ao desenvolvemento, tamén se configurou o repositorio de GitHub. Neste sprint tamén fíxose a inicialización de ambos frameworks e fixéronse probas de funcionamento con Ionic xa que se trataba dun framework novo para o desenvolvedor, rexistrouse a aplicación en Spotify o que permite facer peticións a seu API. Fíxose unha procura de documentación inicial para orientar o proxecto as mellores ferramentas dispoñibles na actualidade.

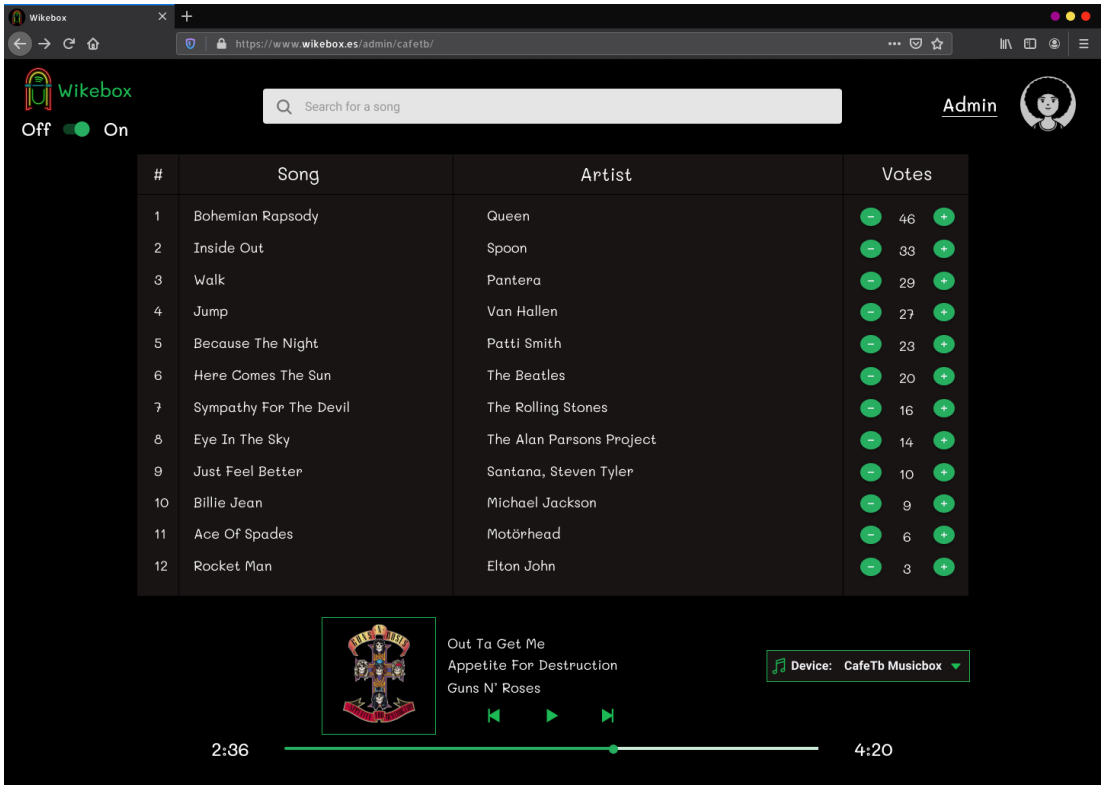
### 4.4.2 Segundo sprint

Nesta fase implementouse o backend, creouse unha aplicación de Django que fai de API, para a comunicación entre os frameworks. Xeneraronse as peticións ao API de Spotify coa axuda da libraría *Spotipy*, tanto as cancións, as procuras coma a obtención dos datos do que se está a reproducir nese intre. Tamén se fixeron as modificacións necesarias para usar o sistema de xestión de usuarios de Django, aínda que tería que ser modificado posteriormente.

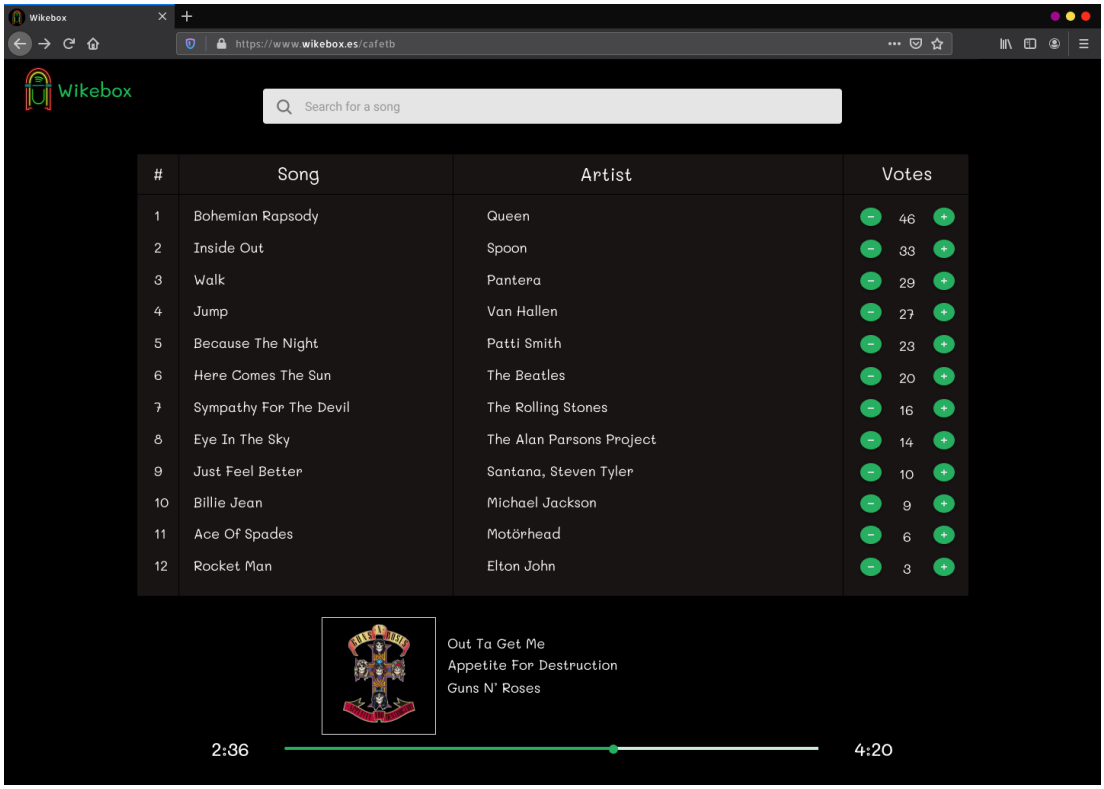
Figura 4.3: Mockups do frontend



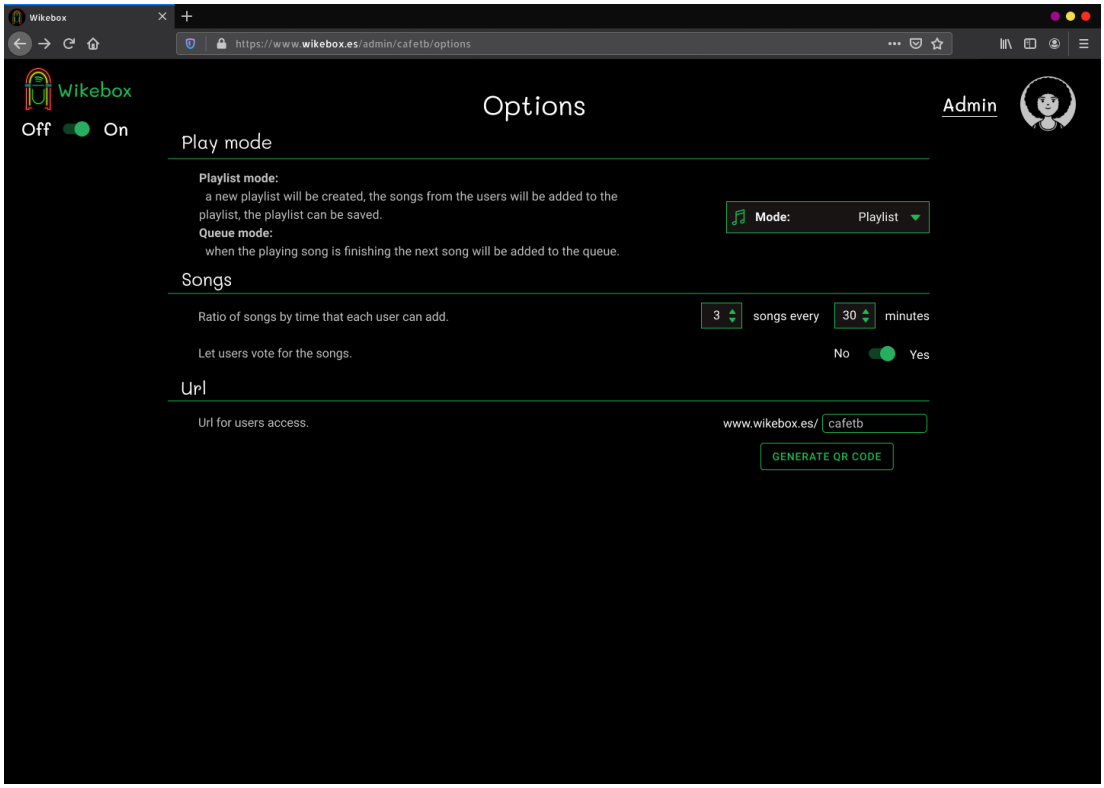

(a) Mockup vista de login




(b) Mockup vista do admin




(c) Mockup vista de usuario




(d) Mockup opcións do admin

Ao finalizar o Sprint comprobouse coa aplicación de Postman 3.1 que o backend respondía as peticións e era capaz de facer modificacións na reprodución de música a través do API.

### 4.4.3 Terceiro sprint

Neste sprint deseñouse a base de datos, fíxose un modelo relacional que se trasladou a aplicación, crearonse os modelos de datos en Django para gardar as sesións, as listas de reprodución das sesións e as cancións que están a soar. Modificouse o modelo xa preestablecido de usuarios en Django para engadir datos de interese coma a sesión a que pertence o usuario e retiráronse outros campos coma o enderezo electrónico, na figura 5.3 podemos ver a súa estrutura. Tamén buscáronse as diferentes alternativas para o almacenamento de datos no frontend.

No Sprint Review comprobouse que era posible o almacenamento dos datos obtidos da plataforma de música e tamén era posible a súa lectura.

### 4.4.4 Cuarto sprint

Para o cuarto sprint se planificou o deseño do frontend baseado nos mockups realizados, crearonse as páxinas adaptándoas o máximo posible ao deseño. Implementouse o servizo no frontend encargado de facer as peticións ao backend. Crearonse as peticións necesarias para a obtención dos datos e se creou a xestión de usuarios tendo en conta os permisos de acceso tanto na parte do frontend coma do backend permitindo tan só as accións necesarias.

Para rematar o sprint comprobouse que o servidor de Ionic era capaz de amosar as diferentes páxinas web e que funcionaban as características implementadas ata o momento.

### 4.4.5 Quinto sprint

O último sprint foi o mais longo debido a dificultade de securizar o sistema, fíxose mediante JWT para a xestión de tokens para os usuarios. Estudáronse outros métodos para facer dita conexión, pero se concluíu que a autenticación con JWT era a mais segura e útil no noso caso. Implementouse un middlware capaz de interceptar as petición e engadir os tokens de acceso, tamén coa posibilidade de renovalos cando é necesario. Por último crearonse as peticións que restaban para a comunicación co backend.

Ao final o Sprint Review comprobouse que a aplicación era funcional e segura, para os usuarios rexistrados non presenta ningunha modificación pero os usuarios non rexistrados non teñen capacidade de facer peticións ao API.

## 4.5 Custos do proxecto

A medición dos custos se fai contemplando entre diferentes categorías os custos técnicos, referentes equipamento hardware e software necesario, custos de servizos, referente aos servizos contratados e custos en recursos humanos. Debido a que non foi necesaria a compra de ningún equipamento nin o aluguer de ningún servidor e tampouco foi necesaria a adquisición de ningún software xa que o utilizado foi gratuíto conclúese que os custos técnicos son nulos, non así os outros.

Para dispor do funcionamento completo da API de Spotify é necesaria a subscrición ao seu servizo *premium* o que ademais ofrece outras vantaxes. Na táboa 2.1 pódese observar os custos dos diferentes plans ofertados, no noso caso utilizouse unha subscrición familiar que resulta nun montante de 2,67 € mensuais e aproximadamente 14 € durante a duración do proxecto.

Para o cálculo do custo dos recursos humanos é necesario coñecer o tempo invertido, desafortunadamente non acadouse a precisión desexada e fíxose unha estimación que podemos observar na táboa 4.2. Do mesmo xeito non é doado impoñer un custo por hora traballada xa que nun proxecto con diferentes recursos humanos existen custos distintos para cada un dos recursos. Decidiuse un prezo de 20 € por hora traballada para realización dos cálculos.

Na seguinte táboa 4.1 podemos ver o custo total aproximado do proxecto.

Táboa 4.1: Custos do proxecto

| Concepto | Custo |
|---|---|
| Spotify premium | 14 € |
| Recursos humanos | 9200 € |
| **Subtotal** | **9214 €** |
| IVE (21 %) | 1934,34 € |
| **Total** | **11148,94 €** |

Táboa 4.2: Tempo empregado no proxecto

| Tarefa | Tempo en horas |
|---|---|
| Documentación | 80 |
| Desenvolvemento do backend | 100 |
| Desenvolvemento do frontend | 100 |
| Conexión e securización | 150 |
| Probas | 30 |
| **Total** | **460** |

Capítulo 5

# Deseño

Partindo dos requisitos e a planificación do proxecto, neste capítulo pásase a explicar os aspectos relevantes do deseño da aplicación, a arquitectura adoptada e os servizos utilizados.

## 5.1 Lóxica de funcionamento

O propósito da aplicación é imitar unha gramola das que habitualmente se atopaban nos bares ou outros negocios da restauración. O deseño desta non quere imitar o aparello físico, se non a súa funcionalidade, adaptada as tecnoloxías de agora, polo tanto de forma virtual, aproveitando tamén para engadir características que melloren a experiencia dos usuarios finais. Partindo deste idea evítase concretar a aplicación nun nicho tan específico, buscando tamén que sexa útil noutros ámbitos nos que se desexe crear unha lista de reprodución compartida sen complicación como pode ser un ambiente de traballo cun fío musical, unha festa privada ou unha reunión de amigos.

Ponse tamén fincapé en que a aplicación resulte sinxela de utilizar tanto para o administrador coma para os usuarios, axudando desta forma a súa adopción. Os pasos desexables para un novo usuario, por exemplo o propietario dun bar, serían os seguintes:

1. Acceder a páxina da aplicación.
2. Creación dunha nova sesión (poderíamos usar o símil dun bar).
3. Creación dun usuario nesta sesión, este ao ser o primeiro será o usuario administrador.
4. Conexión da sesión co usuario de Spotify.
5. Reproducir música en calquera dos clientes de Spotify, xa pode ser a web, os clientes de escritorio ou móbil.

6. Compartir o enlace da sesión para o resto dos usuarios.

No caso dos usuarios novos que acceden a unha sesión xa rexistrada, por exemplo os clientes dun bar, os pasos a reproducir serían os seguintes:

1. Abrir o enlace recibido, este accede a páxina de rexistro de usuarios na nova sesión.

2. Rexistrar o novo usuario (tan só a primeira vez).

3. Procurar e engadir cancións.

4. Votar as cancións favoritas para que suban na cola de prioridade.

Pódese observar que os pasos non son complicados, en canto a outros proxectos similares necesitan de outra aplicación no mesmo sistema para xestionar os usuarios e as colas de reprodución.

## 5.2 Arquitectura

Cando se fala de arquitectura en desenvolvemento software faise unha analoxía coa arquitectura da construción, é un símil xa que o desenvolvemento de software baséase na elección de estruturas fundamentais para a organización do sistema, dita organización inclúe todos os compoñentes, a forma de interactuar uns con outros e co entorno onde se atopan.

Neste proxecto utilízanse dúas arquitecturas diferentes, debido aos frameworks utilizados, aínda que son similares teñen características que os fan distintos. Angular é o framework co que se usa Ionic, utiliza a arquitectura mais coñecida, Model, view, controller (MVC)[1] en cambio Django, o framework usado para desenvolver o backend utiliza Model, template, view (MTV)[2]. Ámbolos patróns de arquitectura baséanse na idea de reutilización do código e separación de conceptos, separa as representacións internas da información e o xeito do que é presentado aos usuarios, isto permite o desenvolvemento paralelo ademais de facilitar a tarefa aos desenvolvedores e facelo mais doado de manter.

### 5.2.1 MVC: Modelo, vista, controlador

Trátase do patrón de deseño software máis coñecido, é moi utilizado no desenvolvemento web. Divide unha aplicación en tres partes interconectadas que coméntanse a continuación:

- **Model**

[1] Modelo, vista, controlador.
[2] Modelo, plantilla, vista.

O **modelo** é a representación da información coa que o sistema opera, polo tanto xestiona todos os accesos a dita información, tanto consultas como actualizacións. Envía a vista a información que esta lle vai solicitando para ser mostrada na interface do usuario, as peticións de acceso ou manipulación chegan ao modelo a través do controlador.

- **View**

  A **vista** encargase da representación de cara ao usuario de toda a información da aplicación, realiza as peticións ao modelo, necesarias para a recuperación de dita información. Proporciona unha experiencia interactiva do lado do cliente para o usuario, o que lle permite interactuar coa aplicación.

- **Controller**

  O **controlador** é o encargado de interpretar as instrucións e accións que realizan os usuarios finais da aplicación, facendo uso das funcionalidades implementadas no modelo, validando os datos e os parámetros das solicitudes recibidas. Pódese dicir que o controlador fai de intermediario entre a vista e o modelo.

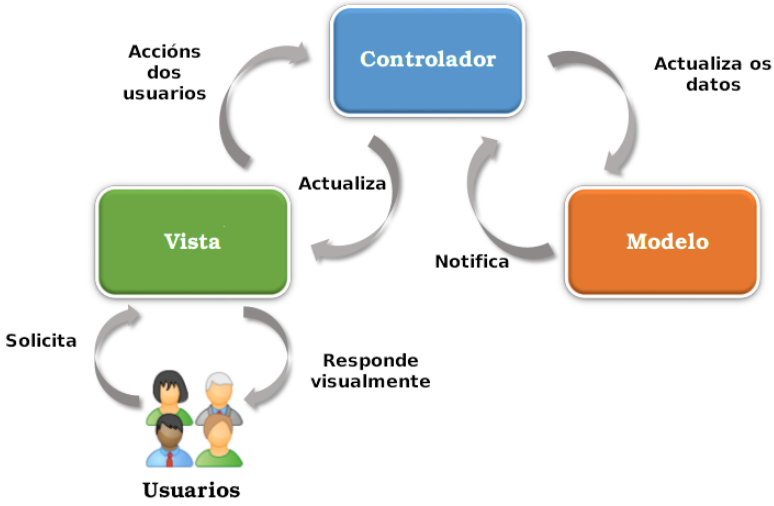



Figura 5.1: Model, controller, view.

### 5.2.2 MTV: Modelo, plantilla, vista

Este patrón é coñecido por usarse en Django, non existe unha gran diferencia entre este patrón e o **MVC** mais ben é unha implementación diferente na que o controlador é manexado directamente polo framework, a continuación explícanse as tres partes que o forman:

- **Model**

  O **modelo** é similar ao caso de MVC, actúa como interface para os datos, é a estrutura lóxica de almacenamento de datos detrás da aplicación web ou a API no noso caso.

- **Template**

  A **plantilla** é o compoñente que marca a diferencia coa arquitectura MVC. A plantilla actúa como a capa de presentación, basicamente é o código HTML que renderiza a información. O contido das plantillas pode ser tanto estático coma dinámico, poderíase comparar a vista na arquitectura MVC.

- **View**

  A **vista** é o enlace entre o modelo e o template, interactúa co modelo e decide que información vai ser amosada e que plantilla renderiza. É similar ao controller explicado na arquitectura MVC.

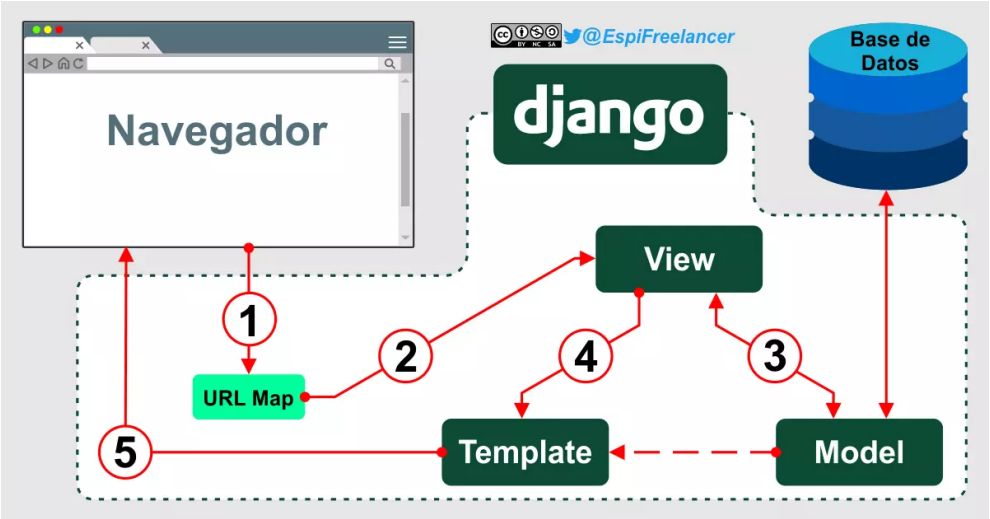



Figura 5.2: Model, template, view.

Como vemos a gran vantaxe das arquitecturas é a separación das responsabilidades dentro da aplicación, deste xeito se existen cambios nos datos (modelo) non afecta a visualización xa que esta se actualizará co contido novo, ademais de que o usuario non interactúa en ningún momento directamente cos datos.

### 5.2.3 Esquema xeral

O esquema xeral para o proxecto é o uso de Ionic para o Frontend e Django rest framework para o Backend:

Decidiuse utilizar Ionic pola súa capacidade de desenvolver con tan só un código tanto unha aplicación web coma aplicacións específicas para os sistemas operativos mais usados nos móbiles hoxe en día, polo uso de linguaxes de programación xa coñecidos, a gran cantidade de elementos para a interface dos usuarios xa xerados e pola súa ampla documentación, ademais do seu cliente que facilita a xestión de novos recursos ou o manexo destes. Aínda que non se xeren as aplicacións móbiles, creemos que é interesante ter esa posibilidade nun futuro para engadir mais características. Sopesouse utilizar tan só Django para todo o desenvolvemento, pero a súa capacidade de adaptación a dispositivos móbiles non é tan boa e perderíamos a posibilidade de crear as aplicacións se así o desexaramos.

No caso de Django decidiuse utilizalo, polo coñecemento previo desta ferramenta, a súa simpleza ao usar Python coma linguaxe de programación, o que simplifica o uso de librerías para a conexión con Spotify por exemplo, tamén é destacable o seu Objects relational mapping (ORM)[3], a interface de administración, a seguridade por defecto e a gran documentación coa que conta.

## 5.3 Frontend

Ionic, usado para o desenvolvemento do frontend, permite o uso de diferentes frameworks para o desenvolvemento, escolleuse utilizar *Angular* xa que é ideal para a creación de Progressive web application (PWA)[4] que é o que pretendemos xerar no proxecto. Aínda que a curva de aprendizaxe é longa, contén outras calidades, o seu alto rendemento, as directivas, o ecosistema ou o uso da libraría de Reactive Extensions Library for JavaScript (RxJS) que está enfocada no patrón de deseño *observer* o que se basea na notificación duns obxectos a outros cando se producen cambios a continuación explícanse os elementos que compoñen este paradigma:

- **Programación reactiva**

  Trátase dun paradigma enfocado ao traballo con fluxos de datos, finitos ou infinitos de forma asíncrona. Ao ser asíncrona a recepción dos datos non produce bloqueos o que facilita a as chamadas a recursos con resposta tardía, coma poden ser os que están en rede, as chamadas a bases de datos, a xestión de peticións e en xeral todo o fluxo de chamadas.

- **RxJS**

[3] Mapeo objeto-relacional
[4] Aplicación web progresiva

É unha librería JavaScript de programación reactiva, facilita a composición de código asíncrono baseado en secuencias de observables. Isto faino grazas a tres patróns de deseño, o patrón observer, iterador e a programación funcional. Angular utiliza de forma nativa esta librería para varias das súas APIs, como por exemplo as chamadas HTTP co módulo *httpClient*. Tamén é usado en outros frameworks coma React, e existen versións desta libraría para outras linguaxes, coma Ruby ou Python.

- **Patrón observer**

  Neste patrón existe un suxeito que é o produtor da información e por outro lado hai un ou varios consumidores desta información. En *RxJS* o suxeito sería o **observable** e o consumidor o **observer**. Os observables son os encargados de propagar a información e notificar os seus cambios, para isto proporciona métodos a partires dos cales os observers poden subscribirse ou cancelar a subscripción aos fluxos de datos.

- **Patrón iterador**

  Permite recorrer contedores de información, coma por exemplo un array, sen necesidade de coñecer o seu contido. *RxJS* proporciona os métodos *hasNext* para indicar se hai mais datos no iterador e *next* para acceder a seguinte posición do iterador.

- **Programación funcional**

  Outro patrón no que se basea *RxJS* é a programación funcional. Un paradigma de sobra coñecido pola súa antigüidade pero que volve a utilizarse grazas a programación reactiva. Na programación funcional o factor mais importante son as funcións, todo está orientado ao uso de funcións e estas permiten o paso de outras funcións coma argumento. As funcións deben ser funcións puras, isto quere dicir que unha función cun mesmo argumento de entrada debe devolver sempre o mesmo resultado. A priori, podemos pensar que isto é unha característica de calquera función pero non é así, por exemplo se se usan variables globais xa depende do elemento externo que pode variar.

### 5.3.1 Arquitectura do frontend

Na sección 5.2, explícanse os diferentes patróns de deseño. Indícase que *Ionic* e *Angular* utilizan o patrón MVC, no caso deste proxecto, ao facer o almacenamento dos datos no backend, non existe un modelo como tal para o almacenamento de datos, aínda que si que almacenamos os tokens de acceso e de refresco para o uso de JWT. A continuación coméntase as singularidades do deseño tanto da Vista coma o Controlador, na sección 5.4.2 explícase o modelo utilizado no backend.

### 5.3.2 Vista

A vista é a última capa da arquitectura do sistema trátase da capa que interactúa co usuario mostrándolle a información e enviando as peticións do usuario ao controlador. Ao tratarse dunha aplicación web, a vista execútase nun navegador, ou nun dispositivo móbil coma PWA, polo tanto son páxinas HTML modificadas por JavaScript, codificado en TypeScript no noso caso grazas a Angular.

A estrutura das vistas do frontend componse das seguintes páxinas:

- **Home**

  Páxina de destino para os usuarios non rexistrados, ao acceder pódese observar unha breve introdución a aplicación e ao seu funcionamento. Dende aquí os usuarios poden acceder a outros lugares de interese, como poden ser o rexistro dunha nova sesión, ou dun novo usuario. No caso dos usuarios conectados non se accede a esta páxina xa que a súa principal función é dar a benvida e proporcionar información.

- **Session**

  Na páxina *session* os usuarios poden rexistrar unha nova sesión para crear un novo espazo de música, o rexistro realízase mediante un formulario no que tan só é necesario introducir o nome desexado para a nova sesión.

- **Register**

  Trátase da páxina ao que dará acceso o enlace de invitación, aquí os usuarios poderán rexistrarse nunha sesión mediante un formulario con un nome de usuario e un contrasinal. Cada sesión ten a súa propia páxina de rexistro xa que unha vez rexistrado o usuario asóciase a dita sesión e é a que poderá conectarse.

- **Login**

  Esta é a páxina que da acceso aos usuarios rexistrados, mediante un formulario introducindo o seu nome de acceso e o contrasinal os usuarios poderán acceder a súa sesión onde poderán engadir cancións e demáis.

- **Music**

  É a páxina principal da aplicación, aquí os usuarios poden procurar cancións e engadilas a cola, tamén poden votarse as cancións favoritas para cambiar a orde de reprodución, ademais amosase información sobre a canción que se está a reproducir nese intre. No caso dos usuarios administradores teñen tamén a posibilidade de seleccionar o dispositivo no que se reproduce a música e escoller unha lista propia da conta de Spotify para

poboar a lista de reprodución da aplicación e así os usuarios teñan a posibilidade de votar pola orde de reprodución.

- **Callback**

  Páxina necesaria para a captura do token do usuario, é a páxina a onde redirixe Spotify despois de facer a autenticación na súa plataforma. Non ten utilidade para o usuario, tan só técnica.

### 5.3.3 Controller

O controlador é a capa intermedia da arquitectura, actúa coma comunicación entre a vista e o modelo. A súa finalidade é a de procesar peticións que chegan dende a vista, xestionalas e comunicarse coa parte do modelo correspondente a petición, unha vez o modelo realiza as operacións, o controlador devolve a vista os datos para amosalos ao usuario. Neste caso a maioría da comunicación realizase no backend, polo tanto as instrucións na parte do controlador do frontend son peticións ao backend. Crearonse tres servizos, para xestionar as peticións, a autenticación e a seguridade nas comunicacións:

- **Api**

  Trátase do servizo principal, é o encargado de realizar as peticións a API do backend. A maioría das funcións xestionase a través de observables, o que permítenos obter a información en canto é actualizada. Debido a que é necesario saber cal é o usuario que está a facer as peticións tódalas funcións son POST xa que envíase a request que contén dita información. As funcións aquí incluídas son todas aquelas que teñen relación coa xestión de Spotify, como poden ser a procura de cancións, a obtención do token de acceso ou a selección do dispositivo para reproducir a música.

- **Auth**

  O servizo de auth manexa a autenticación dos usuarios, utiliza JSON web Token para o control de acceso xa que permite a propagación de identidades e privilexios entre o provedor de información e o solicitante. Para o almacenamento dos JWT utilizase Ionic Storage [2] xa que usa o mellor método para gardar os datos nas distintas plataformas utilizadas.

- **Interceptor**

  Para o uso de autenticación con JWT é necesario engadir o token en cada unha das peticións ao backend *intercept* 6.14 é a función que se encarga de interceptar a petición, posteriormente a función *addToken* **??** é a encargada de engadir o token a cabeceira.

Para garantir a seguridade os tokens de acceso caducan cada 5 minutos polo tanto necesitase renovalos co token de refresco como pódese observar a función *handle401Error* **??** renova o token, se pode, cando se produce un erro de autenticación.

## 5.4 Backend

Deseñado coa idea de publicar noticias dun xornal, Django é un dos frameworks mais populares hoxe en día, segue o principio (DRY, Don't Repeat Yourself) [5] polo que evita a repetición de código innecesariamente. Está escrito en Python, que axuda a súa popularidade, ademais a súa arquitectura destaca pola simpleza a hora de cambiar a forma de visualizar o contido ou de realizar cambios nos datos.

### 5.4.1 Arquitectura do backend

Django utiliza un patrón de deseño específico o **MTV** /refsec:MTV, baseado no patrón *MCV*. Este patrón é lixeiramente diferente, consta de modelo, plantilla e vista, debido a que o frontend manexa a interface de usuario non utilizamos a plantilla xa que é a encargada de renderizar o código para a vista do usuario. A continuación explícase o uso do modelo e da vista neste framework:

### 5.4.2 Modelo

A capa modelo é a parte da aplicación que encapsula a lóxica de negocio e manexa os accesos a base de datos. Nesta capa se almacena, modifican e eliminan os datos que manexa a aplicación. Na figura 5.3 pódense ver as entidades do modelo. A continuación detallase cada unha das entidades.

- **Session**

  Representa os datos das sesións, almacena a lista de sesións que estean rexistradas na aplicación con un nome e unha url de acceso, tamén almacena o nome do usuario administrador da sesión.

- **LocalPlaylist**

  Representa a lista de cancións na cola das sesións, almacena os datos das cancións coma o título, artista e duración, tamén os votos dos usuarios.

- **SongPlayingNow**

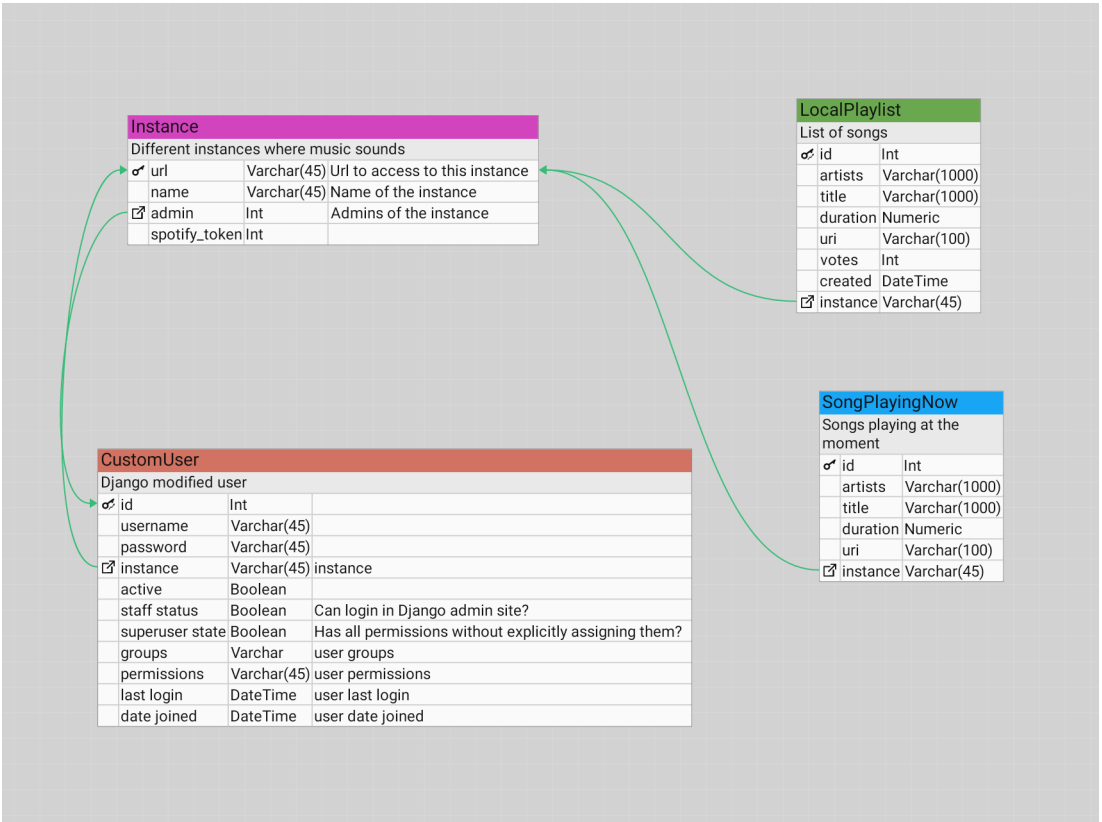
[5] Non te repitas

Representa a canción que se está a reproducir nas sesións, tamén almacena os datos das cancións, utilízase para gardar cal é a canción que esta a soar e facilita engadir a seguinte a cola de reprodución.

- **CustomUser**

  Trátase dunha modificación do modelo *User* predefinido por Django, representa os usuarios da aplicación, retiráronse campos como o correo electrónico e se engadiron outros coma a sesión.

Figura 5.3: Estrutura da base de datos




### 5.4.3 Vista

Na arquitectura MTV é a vista a encargada de executar a lóxica de negocio. Recibe as peticións dende o arquivo *urls.py* e executa as accións necesarias, no noso caso dividimos as peticións entre as necesarias para administrar os usuarios e as relacionadas coa música, dividindo esta última de novo entre as peticións a nosa API e as necesarias para a API de Spotify.

- **User views**

  E conxunto de peticións encargadas do manexo de usuarios, debido a que utilizase o modelo de usuario predefinido por Django con modificacións para adaptarse ao proxecto, tan só tívose que engadir a obtención da sesión do usuario coma podemos ver en 6.5.3.

- **Api views**

  Peticións encargadas de manexar a API propia, coma por exemplo engadir os votos, comprobar se é necesario engadir cancións, poboar a lista de reprodución ...Todas as peticións son de tipo POST debido a que sempre temos que saber cal é o usuario que fai a solicitude.

- **Spotify views**

  E conxunto funcións relacionadas coas peticións a API de Spotify, por exemplo a autenticación de usuarios na plataforma, engadir a seguinte canción a cola, procura de cancións, procura de playlists ...

Capítulo 6

# Implementación

Neste capítulo explícase o proceso de implementación, as decisións importantes durante este, os problemas atopados e as súas solucións.

## 6.1 Estrutura do proxecto

Na seguinte figura 6.1 amósase a estrutura xeral do proxecto e a súa xerarquía de cartafois. Debido a que o proxecto conta con un backend e un frontend existen dúas partes na estrutura ben diferenciadas, primeiro descríbese a estrutura do backend e a continuación a do frontend.

A estrutura principal do backend é similar a dos proxectos *Django*, conta cun proxecto chamado *Wikebackend* e dúas aplicacións, unha chamada *users* que é a encargada da xestión dos usuarios e a outra *api* que é a que xestiona as peticións recibidas e as enviadas a API de Spotify.

- **Wikebackend**

  Garda os arquivos de configuración de Django, entre eles as opcións en *settings.py*. Neste arquivo se almacenan aspectos básicos da configuración, coma son as apps utilizadas, o método de autenticación, o tipo de base de datos, a linguaxe, configuración da zona horaria... Nesta parte da estrutura tamén se almacena a redirección de urls principal no arquivo *urls.py* este redirixe as peticións que recibe aos ficheiros axeitados.

- **Api**

  Trátase da parte principal do proxecto, é a parte encargada de gran parte da lóxica de este, encárgase de xestionar as peticións recibidas e devolver as respostas axeitadas. Tamén contén as peticións necesarias para a iteración coa API de Spotify.

- **Users**




Figura 6.1: Estrutura do proxecto

É a aplicación que xestiona os usuarios, contén as modificacións para a adaptación do modelo de usuarios ao proxecto e as funcións que teñen que ver con estes.

Respecto ao frontend a estrutura segue a distribución xeral dos proxectos en *Ionic* dentro do cartafol *src* atopamos a todo o código do proxecto, descríbese a estrutura a continuación.

- **App**

  Contén toda a lóxica do frontend, ao igual que no backend esta é unha app dentro do proxecto que conta con características únicas dentro deste. Ademais de almacenar as páxinas, tamén contén os servizos, os modelos e os compoñentes utilizados.

- **Assets**

  Neste cartafol gárdanse todos os recursos, tales coma poderían ser clips de son, vídeos ou imaxes, que é o caso do proxecto, deste xeito facilitase a orde e o acceso a este tipo de recursos.

- **Environments**

  Ao igual que moitas outras ferramentas de desenvolvemento, Ionic contén configuracións específicas para o traballo en produción e en desenvolvemento, o que facilita o cambio dun entorno a outro cando se queren facer modificacións, estes arquivos gárdanse no cartafol environments.

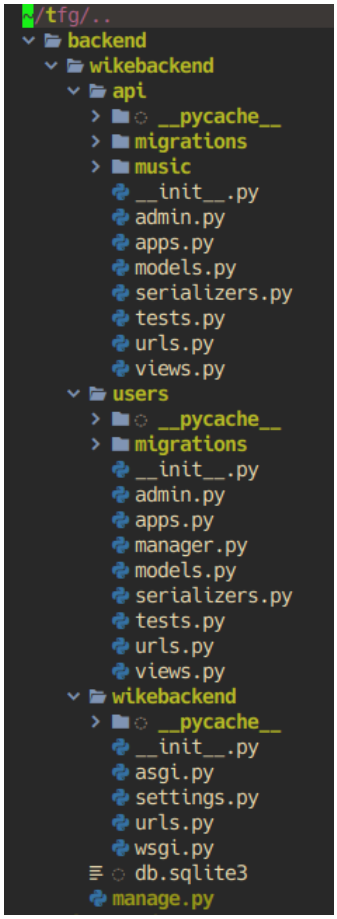



Figura 6.2: Estrutura do backend

- **Theme**

  Almacea as variables correspondentes a folla de estilo, garda as cores principais para axeitar todo o proxecto a un tema en concreto e non ter que introducir as cores por separado en canda sexan necesarias.

A continuación descríbese en profundidade as partes mais extensas que son a aplicación **api** no backend e a **app** no frontend:

**Api** é parte do backend, consta dos arquivos xerais dunha aplicación Django, *urls.py* e o arquivo encargado de xestionar as peticións e executar as funcións de *views.py*. En *models.py* como o seu nome indica defínense os modelos, neste caso as sesións, as cancións que se están a reproducir e as da cola, para facilitar o almacenamento de datos nos modelos utilízanse os serializadores, definidos no arquivo *serializers.py*. Tamén conta cun ficheiro tan só para as peticións a api de Spotify chamado *spotify.py* este é utilizado nas funcións das vistas.

**App** forma parte do frontend, e como xa vimos contén a maior parte do código de este,

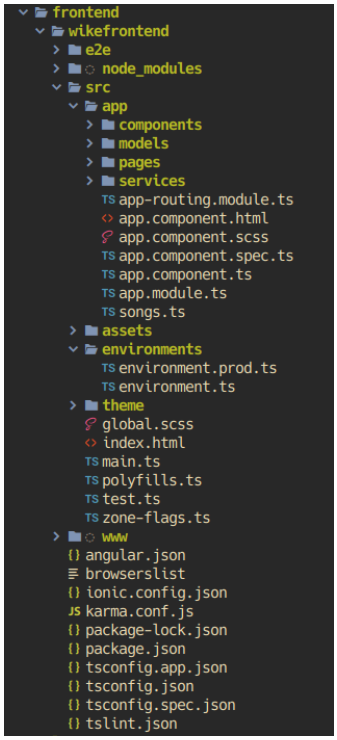



Figura 6.3: Estrutura do frontend

está formado principalmente polas páxinas web as que os usuarios terán acceso, pódense ver na sección 5.3.2, estas contan cos seus módulos e as súas propias follas de estilos.

Para o correcto funcionamento das páxinas son necesarios os servizos, incluídos tamén neste apartado da estrutura, son os principais encargados de realizar as peticións ao backend, están divididos entre os xerais de api, autenticación e o interceptor, encargado de engadir as cabeceiras para o correcto funcionamento dos JWT.

Os compoñentes tamén forman parte de este apartado, son bloques que conteñen funcionalidades específicas, neste caso son utilizados para amosar información en ventás emerxentes é o caso para os resultados das procuras ou a obtención do token de autenticación de Spotify.

Por último tamén se inclúen pequenos modelos utilizados para a transferencia de información, deste xeito dita información pódese xestionar como un bloque, neste caso son utilizados para a autenticación dos usuarios, o almacenamento dos tokens e as cancións.

## 6.2 Percorrido das peticións

As peticións na aplicación seguen un percorrido a través dos ficheiros similar en todos os casos, segundo o tipo de solicitude os servizos e os ficheiros finais que as reciben varían. A continuación explícase o percorrido que seguiría unha procura dunha canción para posi-

blemente ser engadida a lista de reprodución, tamén pódese observar unha infografía desta mesma petición na figura 6.4.

1. Para comezar, o usuario interactúa co frontend en *music.page.html*, neste caso realiza unha procura na barra de procura, ao facer unha modificación no texto actívase o método *searchChanged*.

2. O método atópase en *music.page.ts* que é o controlador da páxina *music*. Fai unha petición ao servizo encargado de xestionar todo o relacionado coa música.

3. A petición chega a *api.service.ts* que encárgase de interactuar co backend, coa axuda dun observable faise unha petición a dirección */api/search/* cando se obteñan os resultados estes serán actualizados.

4. Recíbese no backend a petición, todas pasan polo ficheiro *urls.py* xenérico para posteriormente ser redirixidas ao ficheiro co mesmo nome pero de cada app, neste caso *api*.

5. Unha vez redirixida a petición pásase ao ficheiro *views.py* que contén todas as funcións encargadas de xestionar a api entre elas *search* que fai a solicitude a *spotify.py* o ficheiro que inclúe todas as funcións relacionadas con Spotify.

6. Nesta última función chamada *search* faise a petición a través da libraría *spotipy* ao api de Spotify, o resultado é almacenado nun dicionario de Python.

7. Por último as respostas son recibidas en forma inversa da anteriormente descrita e as cancións obtidas no resultado da procura son amosadas nun cadro listas para ser engadidas na playlist.

## 6.3 Seguridade

Aínda que a aplicación non almacena datos sensibles, xa que o mais relevante é o token de autenticación de Spotify, foi necesario o desenvolvemento dun método seguro para a transferencia de información entre o backend e o frontend. Deste modo evítase que usuarios non rexistrados accedan a aplicación e especialmente conséguese que o usuario administrador teña a posibilidade de acceder a mais opcións sen que accedan a estas o resto de usuarios. A continuación descríbese o proceso de securizar a aplicación xa que conta con diferentes casos e realizouse en diferentes tempos do desenvolvemento.

En calquera comunicación entre un backend e un frontend existe un intercambio de peticións HTTP, cando son realizadas a un dominio, ou porto, diferente ao que pertence a aplicación que está realizando a petición, os navegadores a rexeitan por motivos de seguridade.

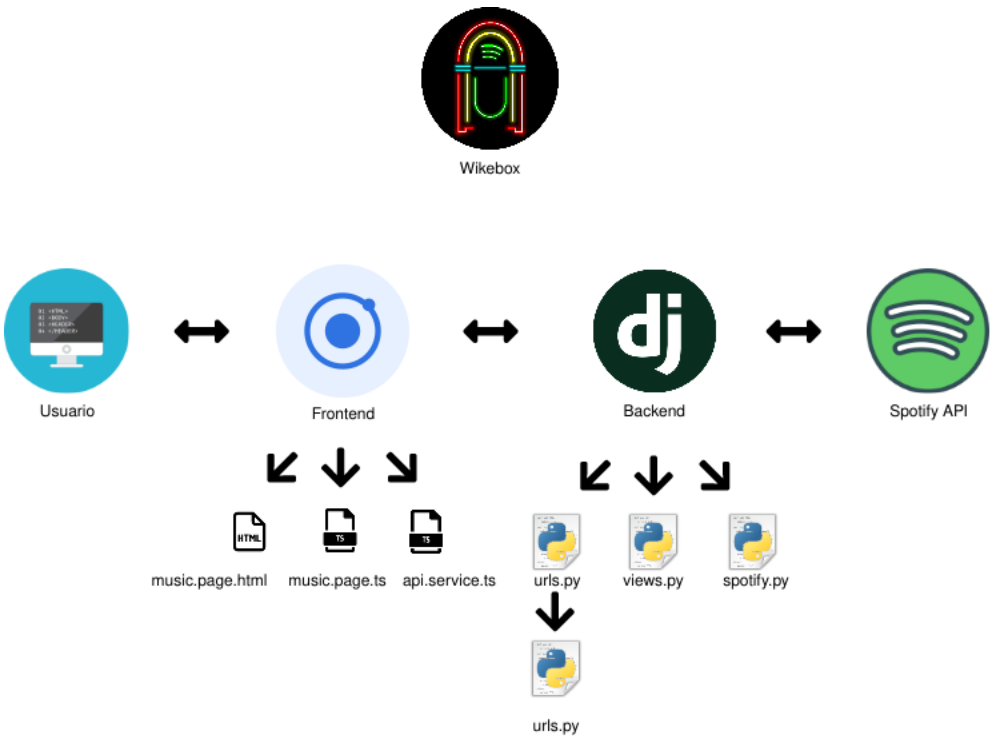



Figura 6.4: Percorrido dunha petición.

Debido a isto é necesario utilizar cabeceiras Cross-Origin Resource Sharing, [1] o seu obxectivo é engadir as cabeceiras HTTP o conxunto de orixes que teñen permiso para ler a información usando o explorador web, deste xeito compróbase que a petición é lexitima e non se está intentando acceder a dominios non coñecidos. Para a modificación das cabeceiras é necesario incluír no middleware de Django a app chamada *corsheaders* [3] que se encarga de facer as modificacións necesarias.

Para comprobar a autenticidade do servidor utilizamos CORS, en cambio para autenticidade dos usuarios é necesario utilizar algún método, preferiblemente que fose o menos intrusivo posible, para isto estudáronse diferentes metodoloxías e posibilidades baseándose nas posibilidades de tanto Django coma Ionic. En Django existen varios métodos preestablecidos para realizar a autenticación:

- **Autenticación básica:** envía tanto o usuario como o contrasinal en texto plano.
- **Autenticación de sesión:** utiliza o propio backend de Django para manter as sesións.
- **Autenticación remota:** permite o uso dun servidor externo que é o encargado de realizar a autenticación.

[1] Intercambio de Recursos de Orixe Cruzado.

- **Autenticación con tokens:** utiliza tokens de acceso e refresco para comprobar que as peticións son lexítimas.

- **Autenticación personalizada:** permite crear un tipo de autenticación propio axeitado para as necesidades exactas.

Tamén existe a opción de utilizar paquetes de terceiros con diferentes métodos de autenticación coma OAuth ou JWT [4].

Estudáronse estes métodos e concluíuse que o mais axeitado sería o uso de sesións ou tokens. A autenticación por sesións non sería axeitada para a comunicación entre o backend e o frontend debido a que está pensada para o seu uso en clientes *AJAX* que funcionan no mesmo contexto que a propia páxina web, polo tanto a elección final é utilizar un sistema de autenticación por tokens. Novamente existen dúas eleccións, o uso de **OAuth** ou **JWT**. Aínda que a autenticación con OAuth podería ser moi axeitada declinouse debido a intención de simplificar o máximo, aínda que cabe destacar que nun futuro sería unha opción a contemplar xa que permitiría unificar as contas de usuarios entre sesións.

Finalmente a opción elixida foi o uso de JWT, existen paquetes de terceiros para facilitar o manexo de JWT con Django, o mais coñecido é *SimpleJWT* [5] que xera un token de acceso con unha caducidade de 5 minutos e un de refresco con caducidade para 24 horas. No caso de Ionic foi necesaria a creación dun *interceptor* que é o encargado tanto de engadir o token as cabeceiras nas peticións, coma de facer a comprobación de se é necesario renovar o token de acceso cada vez que recibe unha nova petición.

O funcionamento é sinxelo, cando se produce unha autenticación satisfactoria no backend este devolve un token de refresco e outro de acceso que é almacenado no frontend, posteriormente cando é necesario facer unha nova petición ao backend estando o usuario xa autenticado o interceptor é o encargado de engadir o token de acceso a dita solicitude. Algúns dos problemas que se tiveron que resolver foron o manexo dos tokens de refresco sen estar o usuario autenticado, xa que é necesario evitar recibir erros de autenticación.

Finalmente para concluír o apartado de seguridade explícase o sistema utilizado para o almacenamento dos tokens no frontend. Debido a que os tokens son transmitidos en texto plano o seu almacenamento non é crítico e non é necesario facer ningún tipo de encriptación, polo tanto os tokens almacenase igual que o resto dos datos na aplicación, utilizando o paquete de Ionic, *storage-angular* que permite gardar os datos na caché do navegador. No caso de que fose necesario Ionic ofrece un paquete premium, de pago, chamado *Ionic secure storage* que permite o cifrado dos datos almacenados utilizando unha base de datos.

## 6.4 Desenvolvemento

Durante o proceso de desenvolvemento seguíronse as ideas acadadas durante o deseño, debido a falta de coñecementos, que se foron acadando ao longo do proceso realizáronse cambios tanto no desempeño das funcións coma na lóxica do funcionamento. Por estas razóns o proceso de implementación foi diferente ao primeiramente deseñado, algo habitual que sucede cando se utiliza unha metodoloxía de desenvolvemento áxil coma é Scrum. A continuación afondarase sobre o desenvolvemento do proxecto comezando pola parte visual e introducindo as partes de código mais relevantes a vez que se detallan os problemas xurdidos e as solucións aplicadas.

### 6.4.1 Frontend

Sendo a primeira idea crear unha aplicación para o funcionamento da gramola, pensouse que para facer mais doado o seu uso e tamén evitar a descarga de aplicacións, era mellor a creación dunha aplicación web. Ademais estas son perfectamente adaptables ao tamaño da pantalla do dispositivo, nas figuras 6.5 podemos ver capturas da páxina de **música** en diferentes dispositivos, a adaptación as diferentes resolucións de pantalla faise mediante Bootstrap xa incluído en Angular. Trátase da páxina na que desexablemente os usuarios van pasar a maior parte do tempo xa que é onde se amosa a información necesaria para o funcionamento e tamén a cola de reprodución, a continuación explícanse os elementos contidos na páxina, cabe destacar que é a vista do administrador xa que inclúe mais opcións.

- **Reproducindo agora**

  Na parte superior esquerda atópase o nome da canción que se está a reproducir nese intre mais o nome do artista. No caso de que non haxa ningunha canción reproducíndose nese intre tamén se indica. A función que permite obter estes datos é un observable que realiza unha petición a API e formatea os resultados utilizando o modelo de *songs*.

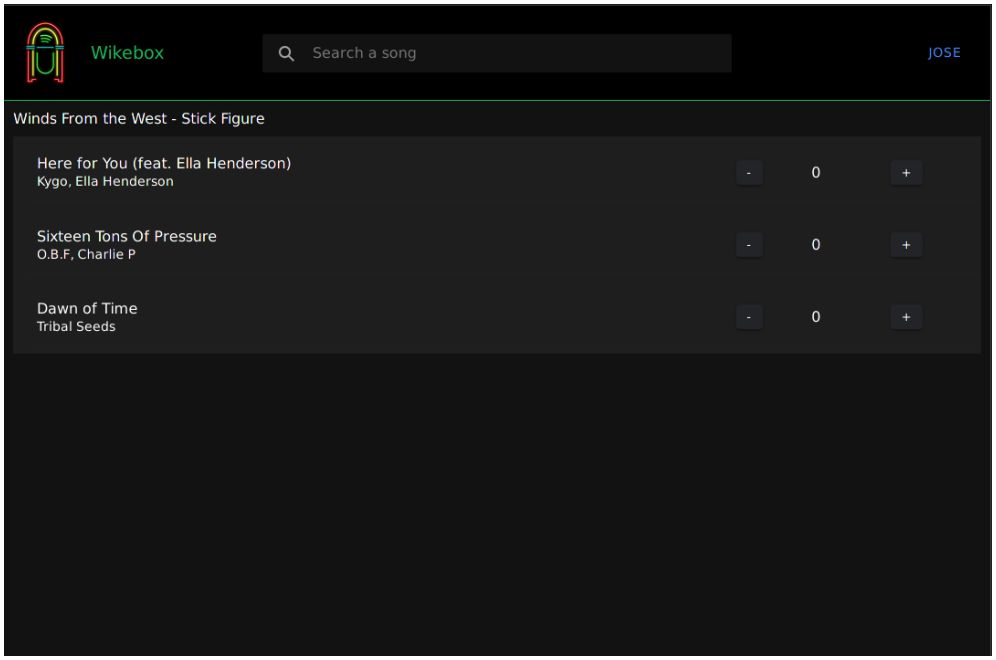
```
1 playingNow(): Observable<IEsongs[]>{
2   return this.httpClient.post<IEsongs[]>(this.SERVER_ADDRESS
      + 'api/playingnow/', {request: Request});
3 }
```

Listing 6.1: api.service.ts

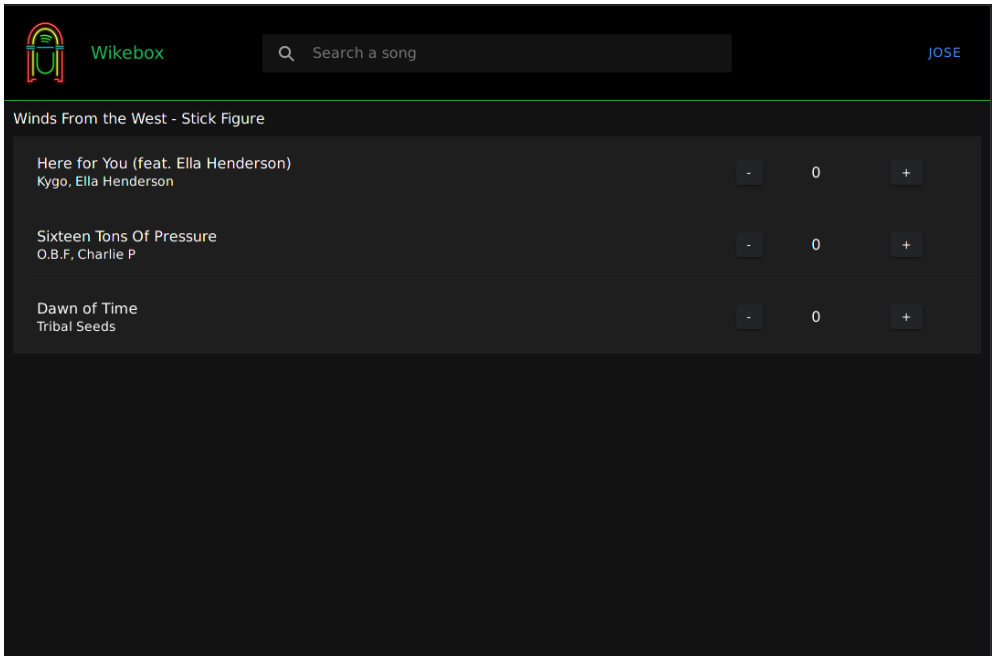
- **Seleción playlist**

  Opción exclusiva para os administradores. Permite seleccionar unha lista de reprodución das gardadas polo usuario na plataforma de Spotify, o propósito é que os usuarios

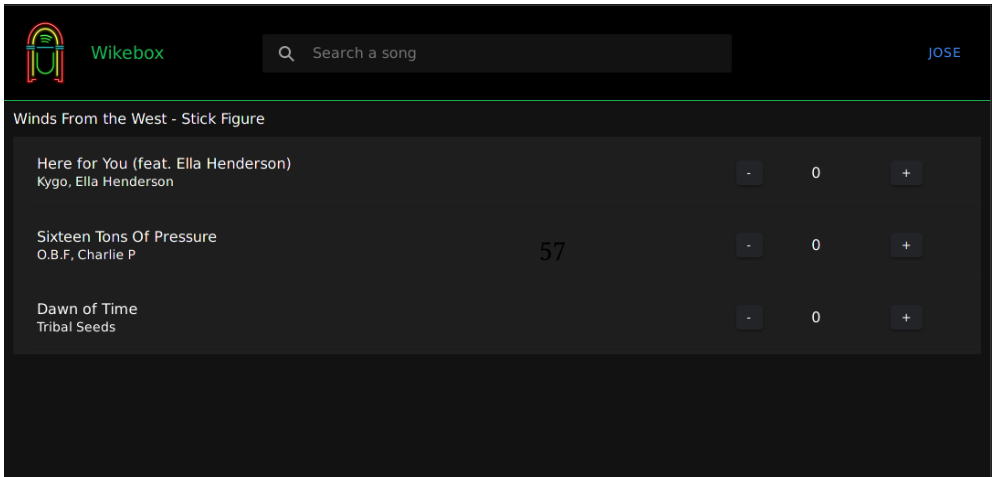
Figura 6.5: Vista en diferentes dispositivos




(a) Smartphone




(b) Tablet



poidan visualizar cancións cando accedan a plataforma sen ter o administrador que engadir cancións de unha en unha, ademais de poder determinar un xénero musical para que os usuarios comecen a facer votacións nas súas cancións favoritas.

- **Selección dispositivo**

  Opción exclusiva para os administradores. Debido a que Spotify é multidispositivo pode darse o caso de que a aplicación se atope aberta en mais dun de eles a vez, o que complicaría o proceso de reprodución xa que por defecto a gramola funciona dende o dispositivo no que se está a reproducir música. Para evitar estes problemas permítese ao administrador seleccionar un dos dispositivos que se atopen conectados nese intre.

- **Barra de procura**

  Ao facer clic sobre a barra pódese escribir un termo a procurar, este termo se transfire a Spotify e se amosan os resultados proporcionados por este, pódese facer unha procura tanto por nome de canción como por nome de artista, quedando excluídos os podcasts xa que considerase que non son necesarios. Amósanse os dez primeiros resultados nun compoñente chamado *ion-popover* que non é mais que un diálogo superposto ao resto da páxina, no que ao facer clic nun destes resultados a canción engádese a lista de reproducción, en 6.2 podemos ver o código que executa o compoñente ao recibir os resultados da procura. O compoñente é creado manualmente e ten un funcionamento similar a unha páxina en HTML como podemos ver en 6.3.

```
async searchShowpop( ev: any ) {
  const popover = await this.searchPopover.create({
    component: SearchresultpopComponent,
    event: ev,
    componentProps: {
      results: this.results,
    },
  });
  // If the search field it's empty don't show the popover
  if (this.searchTerm != "") {
    await popover.present();
  }
  const { data } = await popover.onWillDismiss();
  // Check if there are data
  if (data) {
    this.clickSearchSong( data.item)
  }
}
```

Listing 6.2: musig.page.ts

```
<ion-list>
  <ion-item *ngFor="let item of (results | async);"
    (click)="onClick(item.value.uri)">
    <ion-label>
      <h2>{{ item.value.song }}</h2>
      <h6>{{ item.value.artists }}</h6>
    </ion-label>
  </ion-item>
</ion-list>
```

Listing 6.3: music.page.html

- **Lista de reproducción**

  Trátase da parte da páxina onde se amosan as cancións na orde que se van reproducir, ademais de poder ver o título da canción tamén amosase o artista e ao seu carón o número de votos realizados polos usuarios, isto permite ordenar as cancións segundo a preferencia dos usuarios. Os atributos polos que se ordena esta lista son os seguintes segundo a súa importancia: forma na que se engadiu, número de votos e momento no que se engadiu. Esta lista é gardada na base de datos da API, o que fai que os resultados sexan case instantáneos e se obteñen cun observable. Tamén se inclúe un botón que permite aos usuarios engadir un voto a canción seleccionada.

- **Menú opcións**

  Como o seu nome indica, trátase dun menú depregable onde existen diferentes opcións para os usuarios como pode ser a de pechar a sesión facendo *Logout*.

Unha vez descrita a páxina principal procédese a desgranar as demais, que sen ter tantas opcións conteñen partes moi importantes da aplicación. Tanto a páxina de **session** coma a de **rexistro** son as encargadas de rexistrar unha nova sesión e os novos usuarios. Ámbalas dúas conteñen un formulario de rexistro sinxelo no que hai que rexistrar na sesión tan só hai que introducir o nome de esta e para un usuario tanto o nome de usuario coma un contrasinal. En 6.4 pódese observar o formulario de rexistro de sesións.

```
<form #form="ngForm" (ngSubmit)="session(form)"
    method="post">
  <ion-item style="margin-top:10%">
    <ion-input name="instance" type="text"
    placeholder="Session" ngModel required></ion-input>
```

```
4     </ion-item>
5   </form>
```

Listing 6.4: session.page.html

Aínda que son similares, a páxina de rexistro de usuarios contén unha peculiaridade, os usuarios teñen que estar rexistrados nunha sesión, e en xeral estes van realizar o rexistro a través dun enlace proporcionado cunha forma similar a: `www.wikebox.com/sessionName/register` polo tanto o usuario quedará rexistrado como pertencente a sesión *sessionName*. Para introducir o nome da sesión no rexistro e que non sexa o usuario o encargado de introducilo captúrase directamente dende a url utilizando o método *ngOnInit* como vemos en 6.5. Este método execútase despois de cargar a páxina unha vez Angular finaliza a inicialización do compoñente, garda o nome da sesión e despois o pasa ao formulario de rexistro.

```
1 ngOnInit() {
2   this.mySession =
      this.activatedRoute.snapshot.paramMap.get('mySession');
3 }
```

Listing 6.5: register.page.ts

Como estas funcións pertencen a partes distintas a súa comunicación co backend realizase dende diferentes servizos, o rexistro da sesión se controla a través do *service.api* e o rexistro do usuario dende *auth.service*, en 6.6 pódese ver como se realiza esta última petición.

```
1 register(username: String, password: String,
     mySession:string) {
2   console.log("Welcome: "+username)
3   return this.httpClient.post(this.SERVER_ADDRESS +
      'users/users/', {
4     username: username,
5     password: password,
6     userSession: mySession
7   })
8 }
```

Listing 6.6: auth.service.ts

Respecto a páxina de **login**, de novo é unha páxina sinxela que consta dun formulario no que os usuarios poden acceder a gramola. Do mesmo xeito que no método 6.5 captúrase o nome da instancia para facer a redirección do usuario a súa sesión correspondente. No caso de que o usuario non estea accedendo a aplicación a través da url da súa sesión o que sería `www.wikebox.com/login` é necesario facer unha redirección automática a páxina de home para coñecer a sesión e abrir correspondente, no código 6.7 amosase como é realizado.

```
login(form){
  this.authService.login(form.value.username,
   form.value.password).subscribe(()=>{
    if (this.mySession != null){
      this.router.navigate([this.mySession + '/home'])
    }
    else
      this.router.navigateByUrl('home')
  });
}
```

Listing 6.7: login.page.ts

Dende a propia páxina de login, tamén é posible acceder a páxina de rexistro, tendo en conta o caso de que un novo usuario acceda a esta erroneamente e desexe rexistrarse, neste caso é necesario que o botón redirixa a páxina de rexistro correcta, para o que é necesario coñecer a instancia onde se quere realizar o rexistro a cal pode ser que non estea dispoñible, neste caso notifícase ao usuario que necesita coñecer o nome da sesión.

Probablemente a páxina menos vista polos usuarios sexa a de **callback**, a súa función é a de facer a captura do token enviado por Spotify, polo tanto tan só é útil para os administradores. O funcionamento é completamente automático, unha vez realizada a autenticación na plataforma de Spotify esta redirixe ao usuario a páxina de callback, de novo facendo uso do método *ngOnInit* faise unha petición ao backend para facer o almacenamento do token e posteriormente redirixir ao usuario.

```
ngOnInit() {
    this.apiService.spotifyCallback();
    if (localStorage.getItem("session") != null) {
      this.router.navigate([localStorage.getItem("session")
   + '/home'])
    }
}
```

Listing 6.8: callback.page.ts

Por último falarase sobre a primeira páxina que visualizan os usuarios, trátase da páxina de **home** esta tan só contén as instrucións para que os novos usuarios saiban de que trata a aplicación e brevemente explícase como facer uso desta. A páxina de home tamén é a encargada de realizar diferentes redireccións segundo os usuarios estean xa conectados, se coñeza a súa instancia ou non o código 6.9 amosa como é dita redirección.

```
ngOnInit() {
```

```
2    if (localStorage.getItem("session") != null) {
3      this.router.navigate([localStorage.getItem("session") +
       '/home'])
4    }
5    this.userName = localStorage.getItem("username")
6    this.sessionName = localStorage.getItem("session")
7    if (this.isLoggedIn && localStorage.getItem("session") !=
      null){
8      this.router.navigate([localStorage.getItem("session") +
       '/music'])
9    }
10 }
```

Listing 6.9: home.page.ts

Unha vez rematada a explicación das diferentes páxinas que existen no proxecto, procederase a explicar os servizos necesarios para o resto do funcionamento do frontend.

**Api service** é servizo é o encargado de xestionar todas as peticións que se fan ao backend, todas elas son peticións de tipo *post* debido a que é necesario saber sempre cal é o usuario que está a realizar as peticións, cada unha destas funcións comunicase co backend a través das urls alí definidas e cando reciben a resposta a amosan no lugar correspondente, algunhas destas respostas necesitan ser formateadas tanto para amosalas en lista ou como sexa necesario por exemplo 6.10.

```
1  spotifyChooseDevice(){
2    return this.httpClient
3    .post(this.SERVER_ADDRESS + 'api/choosedevice/',
      {'request': Request})
4    .pipe(
5      map(results => Object
6          .keys(results)
7          .map(key => ({value: results[key]})))
8        ),
9    );
10 }
```

Listing 6.10: api.service.ts

**Auth service** como o seu nome indica, trátase do servizo encargado de xestionar a autenticación dos usuarios no frontend, realizar o login, comprobar os permisos... Aínda que a xestión dos usuarios faise no backend, tanto o rexistro como a xeración dos tokens, é necesario contar con un sistema no frontend para realizar a verificación.

O rexistro realiza unha petición ao backend para a creación dun novo usuario nunha instancia. A función de login utiliza *angular storage* para almacenar os datos necesarios no navegador, estes son o nome do usuario, o token de acceso e o de refresco que son recibidos dende o backend cando se realiza o login pódese ver a función de login en 6.11

```
login(username: String, password: String) {
  this.user = username
  return this.httpClient.post(this.SERVER_ADDRESS +
    'users/token/', {
    "username": username,
    "password": password
  })
  .pipe(
    tap(token => {
      this.storeJwtToken(this.token.access)
      this.storeRefreshToken(this.token.refresh)
      this.storeUser()
      this.storeExpiration()
      .then(
        () => {
          console.log('Token Stored');
        },
        error => console.error('Error storing item', error)
      );
      return token;
    }),
  );
}
```

Listing 6.11: auth.service.ts

Unha vez almacenados estes datos considérase que o usuario está conectado, xa que ao facer a desconexión son borrados, tamén tense en conta que o token de refresco non estea expirado, isto se calcula ao gardalo engadindo un período de validez de 24 horas no código 6.12 pódese ver como se almacena estando o tempo de expiración en milisegundos.

```
private storeExpiration() {
  localStorage.setItem("expiration",
    JSON.stringify(Date.now() + 86400000))
}
```

Listing 6.12: auth.service.ts

Referente a caducidade dos tokens é necesario prestar atención ao servizo de interceptor,

é o encargado de engadir o token as cabeceiras das peticións, pero tamén é o que fai a solicitude de actualizar os tokens xa expirados para os usuarios que aínda están conectados. O token de acceso ten unha caducidade de 5 minutos e para renovalo é necesario ter o token de refresco, facendo unha petición con este token obtense un novo de acceso, este procedemento é invisible para o usuario a función encargada de isto é *refreshToken* 6.13.

```
refreshToken() {
  return this.httpClient.post(this.SERVER_ADDRESS +
   'users/token/refresh/', {
    'refresh': this.getRefreshToken()
  }).pipe(tap((tokens: AuthResponse) => {
    this.storeJwtToken(tokens.access);
  }));
}
```

Listing 6.13: auth.service.ts

Aínda que esta é so unha das funcións deste servizo é unha das mais complicadas xa que é necesario permitir a solicitude de un token de acceso a usuarios que non teñen acceso xa, tan só teñen o token de refresco e isto dificulta o funcionamento. A función que intercepta as peticións é 6.14 esta comproba todas as peticións HTTP e engade o token cando é posible, no caso de recibir un erro de autenticación 401 chama a función *handle401Error* 6.15 que trata de obter novamente o token de acceso e se é satisfactorio realiza novamente a petición.

```
intercept(request: HttpRequest<any>, next: HttpHandler):
   Observable<HttpEvent<any>> {
  if (this.authService.getJwtToken()) {
    request = this.addToken(request,
    this.authService.getJwtToken());
    console.log(request)
  }
  return next.handle(request).pipe(catchError(error => {
    if (error instanceof HttpErrorResponse && error.status
    === 401) {
      return this.handle401Error(request, next);
    } else {
      return throwError(error);
    }
  }));
}
```

Listing 6.14: interceptor.service.ts

```
1  private handle401Error(request: HttpRequest<any>, next:
       HttpHandler) {
2    if (!this.isRefreshing) {
3      this.isRefreshing = true;
4      this.refreshTokenSubject.next(null);
5
6      return this.authService.refreshToken().pipe(
7        switchMap((token: any) => {
8          this.isRefreshing = false;
9          this.refreshTokenSubject.next(token.access);
10         return next.handle(this.addToken(request,
       token.access));
11       }));
12
13   } else {
14     return this.refreshTokenSubject.pipe(
15       filter(token => token != null),
16         take(1),
17       switchMap(access => {
18         return next.handle(this.addToken(request, access));
19       }));
20   }
21 }
```

Listing 6.15: interceptor.service.ts

### 6.4.2 Backend

Unha vez introducido o funcionamento do frontend pásase a explicar de forma similar o funcionamento do backend. Como xa explicouse, contén dúas aplicacións, unha encargada do control de usuarios chamada *users* e outra chamada *api* que é a encargada de facer o resto de funcións, tanto as peticións a API de Spotify como as modificacións dos datos no modelo etc.

O modelo é a estrutura onde se almacean os datos da aplicación, Django por defecto utiliza *SQLite* como base de datos, decidiuse manter esta librería para o manexo dos datos xa que as suas características principais son a eficiencia, simplicidade e independencia, o que quere dicir que é axeitado para sistemas pequenos onde non se reciben moitas peticións diarias ademais ten un pequeno uso de recursos. Existen catro táboas diferentes no modelo unha é a que xestiona os usuarios rexistrados e as outras tres serven para o manexo dos datos, comezaremos explicando estas últimas.

- **Session** almacea as diferentes sesións que os usuarios poden crear para reproducir música en conxunto. Contén un identificador único que é o nome da sesión, este tamén se utiliza na url para que os usuarios poidan acceder, un nome modificable, a ruta onde se garda o token de acceso a Spotify e como chave foránea o nome do usuario administrador.

```
1 class session(models.Model):
2     url = models.CharField(max_length=1000, primary_key=True)
3     name = models.CharField(max_length=100)
4     spotify_token = models.CharField(max_length=1000,
    null=True, blank= True)
5     admin = models.ForeignKey("users.CustomUser",
    on_delete=models.DO_NOTHING, null=True, blank=True)
6     def __str__(self):
7         return self.name
```

Listing 6.16: api/models.py

- **LocalPlaylist** garda as diferentes listas de reprodución que existen nas sesións, está relacionada con estas a traves dunha chave foránea, contén para cada canción os datos típicos de estas como son o o título, o artista e a duración, tamén almacea o identificador único en Spotify chamado *uri*. Os demáis campos que se almacean serven para realizar a ordenación das cancións dentro da lista, primeiro por votos, depois pola forma de ser engadida, isto quere dicir se foi cargada dende unha lista ou engadida a través da búsqueda e por último a órde cronolóxica.

```
1 class LocalPlaylist(models.Model):
2     id = models.BigAutoField(primary_key=True)
3     artists = models.CharField(max_length=1000)
4     title = models.CharField(max_length=1000)
5     duration = models.DurationField(default=None)
6     uri = models.CharField(max_length=200)
7     votes = models.IntegerField(default=0)
8     created = models.DateTimeField(default=datetime.now,
    blank=True)
9     added_mode = models.CharField(max_length=50)  # Added
    during search or during playlist load
10    session = models.ForeignKey(session,
    on_delete=models.DO_NOTHING)
11    def __str__(self):
12        return self.title
```

Listing 6.17: api/models.py

- **SongPlayingNow** o outro modelo que utilízase para a a xestión das cancións almacea o estado da sesión, garda a canción que se está a reproducir nese intre, esto serve para poder xestionar a cola de reprodución de Spotify xa que a API de este non nos permite consultala nin borrar os elementos de esta, tan só engadir novas cancións, polo tanto é necesario saber cal é a canción que se está reproducindo para engadir a seguinte no momento axeitado. O modelo é unha reducción do anterior, tan só garda os datos da canción e cunha chave foránea relacionase coa sesión a que pertence.

- **CustomUser** Django contén un sistema de xestión de usuarios, aínda que é bastante completo como é lóxico non é posible que se adapte por completo as necesidades de cada proxecto, por isto permíte facer modificacións sobre este para engadir campos ou quitalos, deste xeito o funcionamento da xestión de usuarios está creada e tan só é necesario adaptalo. Neste caso retirouse o campo de email, xa que non é necesario e engadiuse o de sesión para poder facer as relacións, ademais cambiouse a chave primaria ao nome do usuario xa que antes este atributo pertencía ao email.

Sabendo agora como é a implementación do modelo, pásase a explicar o funcionamento lóxico do backend, en Django este código almacéase nas chamadas *views* que reciben as peticións dende o arquivo *urls*. Existe un aquivo *urls* para todo o backend que posteriormente desvia as peticións a cada un dos arquivos *urls* de cada unha das aplicacións pódese ver na figura 6.4.

A aplicación de Django encargada de xestionar os usuarios contén as modificacións mínimas, xa que proporcionase unha lóxica xa creada e adaptada para ser modificada. Engadiuse a libraría *Rest Framework Simplejwt* que é a encargada de xerar os tokens de acceso para os usuarios.

A outra aplicación que exite chamada *api* é a encargada de todo o funcionamento da gramola, inclúe todas as peticións para a xestión das canción e o manexo destas a través da API de Spotify. Os pasos principais que se executan son os seguintes:

1. Comprobar se falta mais de 30 segundos para que remate a canción que está soando, función *addToQueue* 6.5.3.

2. Se faltan mais, non se fai nada.

3. Se non, a seguínte canción que está na lista de reprodución engadese a cola de Spotify.

4. Compróbase se a canción que está a soar é a mesma que a primeira na lista de reprodución, función *popTop* 6.5.3.

5. Se non, non se fai nada.

6. Se sí, borrase a canción da lista de reprodución.

Deste xeito conséguese utilizar a gramola sen interromper o uso normal de Spotify xa que o usuario que teña acceso pode seguir utilizandoa como de costume tan só vese afectado cando faltan menos de 30 segundos para o remate da canción.

```
1  lastInQueue = None # Global var for keep the last song added
       to the queue
2  def addToQueue(wike):
3      totalTime =
       SongPlayingNow.objects.filter(instance=wike).values('duration').first()
       # Get the total duration of the song
4      nowTime = spotify.currentProgress(wike) # Get the
       progress of the song
5      if (totalTime != None) and (nowTime != None): # Check
       that there are a song playing
6          totalTime = totalTime['duration']
7          nowTime =
       timedelta(milliseconds=nowTime['progress_ms'])
8          changeTime = nowTime + timedelta(seconds=30) #
       Establish the time to add the next song
9          if changeTime > totalTime:
10             adding =
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').values('uri').first()
11             global lastInQueue
12             if adding != lastInQueue: # Check if the last
       added song is the same as the one at the top of the
       LocalPlaylist
13                 spotify.addToQueue(wike)
14                 lastInQueue =
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').values('uri').first()
15                 res = "Change"
16             else:
17                 res = "Already added"
18         else:
19             res = "No change"
20     else:
21         res = "No playing"
22     return res
```

Listing 6.18: api/views.py

```
def popTop(wike):
    current =
    SongPlayingNow.objects.filter(instance=wike).values('uri').first()
    top =
    LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
    '-added_mode', 'created').values('uri').first()
    if current != None:
        if current == top:
            res = "same"

    LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
    '-added_mode', 'created').first().delete()
        else:
            res = "notSame"
    else:
        res = "Not playing"
    return res
```

Listing 6.19: api/views.py

Unha das características que diferencian esta aplicación de outras similares é a de non necesitar ningún sistema a parte que se encargue de facer as comprobacións, engadir cancións a cola e demáis, isto é grazas a función *checkerThread* esta encárgase de facer unha comprobación cada 10 segundos en cada unha das sesións que están a reproducir música que comproba se é necesario engadir unha nova canción a cola, para isto é necesaria a creación dun fío paralelo xa que de outro xeito non sería posible admitir mais peticións porque dita función bloquearía o resto, pódese ver o código da función en 6.5.3.

```
def checkerThread():
    while True:
        wikes = instance.objects.all()
        for wike in wikes:
            if spotify.areToken(wike):
                if spotify.currentProgress(wike) != None and
    spotify.currentProgress(wike)['is_playing']:
                    addToQueue(wike)
                    popTop(wike)
        time.sleep(10)

x = threading.Thread(target=checkerThread)
x.start()
```

Listing 6.20: views.py

## 6.5 Backend

Aínda que foi deseñado coa idea de publicar noticias dun xornal, Django é un dos frameworks mais populares hoxe en día, segue o principio (DRY, Don't Repeat Yourself) [2] polo que evita a repetición de código innecesariamente. Está escrito en Python, que axuda a súa popularidade, ademais como xa explicamos utiliza o a arquitectura **MTV** que destaca pola simpleza a hora de cambiar a forma de visualizar o contido.

### 6.5.1 Modelo

A capa modelo é a parte da aplicación que encapsula a lóxica de negocio cos accesos a base de datos. Nesta capa se almacea, modifica e eliminan os datos que manexa a aplicación. Na figura 5.3 pódense ver as entidades do modelo. A continuación detallase brevemente cada unha das entidades.

- **Instance:** representa os datos das sesións, almacenase a lista de sesións que estean rexistradas na aplicación.
- **LocalPlaylist:** representa a lista de cancións na cola das sesións.
- **SongPlayingNow:** representa a canción que se está a reproducir nas sesións.
- **CustomUser:** representa os usuarios da aplicación, é unha modificación do modelo User de Django adaptado para o proxecto

### 6.5.2 Plantilla

Debido a que utilizamos Django coma backend non é necesario o uso da plantilla para amosar os datos, esta responsabilidade recae no Frontend.

### 6.5.3 Vista

Na arquitectura MTV é a vista a encargada de executar a lóxica de negocio. Recibe as peticións dende o arquivo *urls.py* e executa as accións necesarias, no noso caso dividimos as peticións entre as necesarias para administrar os usuarios e as relacionadas coa música, dividindo esta última de novo entre as peticións a nosa API e as da API de Spotify.

[2] Non te repitas

### User views:

debido a que utilizamos o modelo de usuario de Django modificado é este o encargado do manexo de case todas as peticións, nos tan só tivemos que engadir a obtención da sesión do usuario coma podemos ver en 6.5.3.

```
@api_view(['POST'])
def getInstance(request):
    user = CustomUser.objects.filter(username=request.data['username']).values()
    userInstance = user[0]['userInstance_id']
    return Response(data=userInstance, status=200)
```

### Api views:

as funcións explicadas a continuación son algunhas das mais destacables das solicitudes que se fan a xestión da nosa API, todas as peticións son de tipo POST debido a que sempre temos que saber cal é o usuario que fai a solicitude.

- **PopTop** é a función encargada de borrar a primeira canción da lista se é a que está a soar.

```
def popTop(wike):
    current = SongPlayingNow.objects.filter(instance=wike).values('uri').first()
    top = LocalPlaylist.objects.filter(instance=wike).order_by('-votes', '-added_mode', 'created').values('uri').first()
    if current != None:
    if current == top:
        res = "same"

    LocalPlaylist.objects.filter(instance=wike).order_by('-votes', '-added_mode', 'created').first().delete()
    else:
        res = "notSame"
    else:
        res = "Not playing"
    return res
```

- **AddToQueue** engade a seguinte canción a cola de reprodución de Spotify, comproba que falten menos de 30 segundos da canción que está a soar e engade a seguinte canción

da lista local de cancións. Utiliza unha variable global xa que é necesario saber cal foi a última canción engadida para non engadila mais dunha vez.

```
1  lastInQueue = None # Global var for keep the last song added
       to the queue
2  def addToQueue(wike):
3      totalTime =
       SongPlayingNow.objects.filter(instance=wike).values('duration').first()
       # Get the total duration of the song
4      nowTime = spotify.currentProgress(wike) # Get the
       progress of the song
5      if (totalTime != None) and (nowTime != None): # Check
       that there are a song playing
6          totalTime = totalTime['duration']
7          nowTime =
       timedelta(milliseconds=nowTime['progress_ms'])
8          changeTime = nowTime + timedelta(seconds=30) #
       Establish the time to add the next song
9          if changeTime > totalTime:
10             adding =
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').values('uri').first()
11             global lastInQueue
12             if adding != lastInQueue: # Check if the last
       added song is the same as the one at the top of the
       LocalPlaylist
13                 spotify.addToQueue(wike)
14                 lastInQueue =
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').values('uri').first()
15                 res = "Change"
16             else:
17                 res = "Already added"
18         else:
19             res = "No change"
20     else:
21         res = "No playing"
22
23     return res
```

- **VoteP** é a función de votación, permite ao usuario engadir un voto novo a canción elixida.

```
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def voteP(request):
    N = LocalPlaylist.objects.values_list('votes',
   flat=True).get(instance=request.user.userInstance,
   uri=request.data['uri'])
    N = N+1

   LocalPlaylist.objects.filter(instance=request.user.userInstance).filt
    return Response(data="plus", status=200)
```

- **CheckerThread** para implementar a funcionalidade de que o administrador non teña que ter a páxina da aplicación aberta e as comprobacións para detectar se a canción vaise acabar ou se ten que borrar a seguinte, creamos unha función que executa ditas comprobacións cada 10 segundos. Esta función execútase nun fío paralelo, xa que Django ten que poder responder ao resto das peticións e desta forma evitamos o bloqueo.

```
def checkerThread():
    while True:
        wikes = instance.objects.all()
        for wike in wikes:
            if spotify.areToken(wike):
                if spotify.currentProgress(wike) != None and
   spotify.currentProgress(wike)['is_playing']:
                    addToQueue(wike)
                    popTop(wike)
        time.sleep(10)

x = threading.Thread(target=checkerThread)
x.start()
```

### Spotify views:

o conxunto seguinte de funcións son as relacionadas coas peticións a API de Spotify, algunhas son simplemente a petición directamente, por isto só amosamos as mais destacables.

- **Amanager** esta función é a encargada de xerar o xestor de autenticación de Spotify.

```
def amanager(userInstance):
    instance = str(userInstance)
    auth_manager =
   spotipy.oauth2.SpotifyOAuth(client_id=client_id,
```

```
    client_secret=client_secret, redirect_uri=redirect_uri,
    scope=scope, cache_path=".spotify_token_" + instance)
    return auth_manager
```

- **ChooseDevice** permite ao administrador elixir en que dispositivo quere que se reproduza a música no caso de que haxa mais de un.

```
def chooseDevice(userInstance):
    client = getClient(userInstance)
    devices = client.devices()
    devices = devices['devices']
    devicesDict = {}
    # If no active devices send no active devices
    if len(devices) >= 1:
        for i in range(len(devices)):
            devicesDict[i] = {
                    'id': devices[i]['id'],
                    'name': devices[i]['name']
                    }
    else:
        devicesDict[0] = {
                'id': "No id",
                'name': "No active devices"
                }

    devicesJson = json.dumps(devicesDict)
    return devicesJson
```

- **AddToQueue** engade a seguinte canción a cola de reprodución de Spotify.

```
def addToQueue(userInstance):
    client = getClient(userInstance)
    curi =
    LocalPlaylist.objects.filter(instance=userInstance).order_by('-added_mode',
    '-votes', 'created').values('uri').first() # Get the uri
    of the next song
    ccuri = json.dumps(curi['uri'])
    cccuri = ccuri[15:]
    rcccuri = cccuri.replace("\"", "")
    try:
        client.add_to_queue(rcccuri, None)
    except:
        pass
```

- **Search** fai a procura das cancións para o usuario.

```
    def search(search, userInstance):
        client = getClient(userInstance)
        searchResponse = client.search(search, limit=10) # Retuns a 'dict' of songs
        cleanlist = searchResponse['tracks']['items'] # Gets the important part of the response
        # Creates a dictionary and populates it with the responses
        songList = {}
        for i in range(len(cleanlist)):
            songList[i] = {
                    'song': cleanlist[i]['name'],
                    'artists': cleanlist[i]['artists'][0]['name'],
                    'uri': cleanlist[i]['uri']
                    }
        songJson = json.dumps(songList) # Convert the dictionary to a json
        return songJson
```

- **SearchToLocalPlaylist** engade a canción seleccionada ao modelo de datos.

```
def searchToLocalPlaylist(songUri, userInstance):
    client = getClient(userInstance)
    playlist = LocalPlaylist()  # Assign the model
    searchedSong = client.track(songUri)
    playlist.pk = None  # The pk is assigned automatically, this is needed for that
    artists = searchedSong['artists']  # Loop in the artist if there are more than one
    string = ""
    for j in range(len(artists)):
        string += artists[j]['name']
        if j+1 != len(artists):
            string += ", "  # Separate the artist names
    playlist.artists = string
    playlist.title = searchedSong['name']
    playlist.duration = datetime.timedelta(milliseconds = searchedSong['duration_ms'])
    playlist.uri = searchedSong['uri']
    playlist.created = datetime.datetime.now()
```

```
17      playlist.added_mode = "search"
18      playlist.instance = userInstance  # The instance of the
        user comes as parameter
19      playlist.save()
20      return json.dumps(searchedSong)
```

## 6.6 Instruccións de execución

Capítulo 7

# Conclusións e traballo futuro

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras.

## 7.1 Conclusións

O sistema creado cumpre os requisitos desexados, creamos unha gramola virtual que permite aos usuarios o uso mais básico de estas, que é a reprodución das cancións preferidas nunha orde. Ademais de ser un sistema o mais sinxelo posible o que facilita o uso xa que con poucas iteracións do usuario pódese crear unha sesión e reproducir música en calquera dispositivo con Spotify de forma colaborativa.

## 7.2 Traballo futuro

Coma calquera proxecto software, sempre cabe a posibilidade da implementación de novas ideas que melloren o produto final, no noso caso a lista podería ser moi longa xa que existen moitos sistemas diferentes para a reprodución de música e a inclusión de algúns destes melloraría o valor da nosa aplicación. Nun principio a aplicación non busca o lucro, aínda que se poderían poñer en funcionamento algunhas funcionalidades para que así fose. A continuación expoñemos algunhas das ideas que se poderían desenvolver para a mellora da aplicación, estas son sempre opcións non exclusivas, polo que cun panel de axustes o administrador podería elixir a súa configuración desexada.

- **Modo DJ:** cun funcionamento similar, a aplicación non engadiría as cancións automaticamente a cola, mostraría unha lista a unha persoa que se dedique a pinchar segundo os votos dos usuarios e as súa propia elección.

- **Sistema de créditos:** limitar o número de cancións por usuario e ofrecer un código para conseguir aumentar o límite, por exemplo nun negocio de restauración para incitar o consumo dos clientes.
- **Usuarios globais:** poder ter o mesmo usuario para diferentes sesións en diferentes lugares. Deste xeito poderíase manter un avatar e unhas estadísticas de uso persoais.
- **Estadísticas:** amosar aos usuarios estadísticas da sesión, cancións mais reproducidas, usuarios que mais cancións engaden...
- **Gardar listas:** as cancións que soen nun período de tempo gardalas nunha lista de reprodución de Spotify, deste xeito poderíase gardar por exemplo a lista de reprodución dunha noite nun bar.
- **Escoller estilos musicais:**
- **Oauth:** acceder con facebook, google...
- **Aplicación iOS, Android:** posibilidade de compartir directamente dende Spotify un enlace a unha canción.

# Apéndices

# Relación de Acrónimos

**CORS** Cross-Origin Resource Sharing.

**CSS** Cascading Style Sheets.

**HTML** HyperText Markup Language.

**HTTP** Hypertext Transfer Protoco.

**IDE** Integrate Development Environment.

**JWT** JSON web Token.

**MTV** Model, template, view.

**MVC** Model, view, controller.

**ORM** Objects relational mapping.

**PWA** Progressive web application.

**RxJS** Reactive Extensions Library for JavaScript.

**UML** Linguaxe unificado de modelado.

# Glosario

**Angular** Framework para aplicacións web desenvolvido en TypeScript, de código aberto e mantido por Google, utilizase para crear e manter aplicacións web dunha soa páxina.

**API** Conxunto de definicións e protocolos que se utiliza para desenvolver e integrar software das aplicacións.

**backend** Parte do desenvolvemento web que se encaga da lóxica para que as aplicacións funcionen.

**django** Framework web de alto nivel en Python que facilita o desenvolvemento rápido e limpio.

**framework** Entorno de traballo con un conxunto de conceptos, prácticas e criterios estandarizado para resolver un tipo de problemática particular.

**frontend** Parte da aplicación que interactúa cos usuarios, é coñecida coma o lado do cliente.

**ionic** Conxunto de ferramentas para desenvolver aplicacións de alta calidade e multiplataforma.

**Scrum** Proceso no que se aplica de maneira regular un conxunto de boas prácticas para traballar colaborativamente e obter o mellor resultado posible dun proxecto.

**Spotify** Servicio dixital de reprodución de música, podcast e vídeos.

**SQLite** Sistema de xestión de bases de datos relacional compatible con ACID, contida nunha relativamente pequena biblioteca escrita en C, é un proxecto de dominio público.

# Bibliografía


[1] [Online]. Available: https://httpie.io/

[2] [Online]. Available: https://github.com/ionic-team/ionic-storage

[3] [Online]. Available: https://github.com/adamchainz/django-cors-headers

[4] [Online]. Available: https://www.django-rest-framework.org/api-guide/authentication/#third-party-packages

[5] [Online]. Available: https://django-rest-framework-simplejwt.readthedocs.io/en/latest/